FINANCIAL ENGINEERING

KBP FFÚ & KSTP FIS

Case Studies in Capital Markets and Banking

Introduction to R Packages

Ing. Milan Ficura, Ph.D.
milan.ficura@vse.cz

EVROPSKÁ UNIE
Evropské strukturální a investiční fondy
Operační program Výzkum, vývoj a vzdělávání

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

Section 2:
R Packages

R packages

- **Introduction to R packages**
- Introduction to Tidyverse
- Introduction to Tidyquant

R packages – Introduction

- **R packages**
  - User-build libraries that extend the functionalities of R
  - There are currently over 15 000 R packages in the **CRAN database** (May 2020)
  - CRAN repository requires some minimum package quality standards + detailed documentation
  - Packages can be found also in other repositories such as **GitHub**, **GitLab**, etc.
- **Working with packages:**
  - **Install R package from CRAN:** `install.packages("PACKAGE")`
  - **Load R package:** `library(PACKAGE)`
  - **List of loaded packages:** `sessionInfo()`
  - **Call FUN from unloaded package:** `PACKAGE::FUN`
  - **Call help for NAME:** `?NAME`
  - **More detailed help for NAME:** `??NAME`
- To install non-CRAN packages, we can use the package **devtools**:
  - **Install the devtools package:** `install.packages("devtools")`
  - **Load the devtools package:** `library(devtools)`
- We can then install packages from GitHub and GitLab as follows:
  - **Install package from GitHub:** `install_github("AUTHOR/NAME")`
  - **Install package from GitLab:** `install_gitlab("AUTHOR/NAME")`

Some basic R packages (1)

- **Data processing:**
  - **tidyverse** → Collection of packages for data processing, cleaning and visualization
  - **data.table** → Package especially usefull for working with very large datasets
- **Time series analysis:**
  - **zoo** → Package usefull for time-series processing
  - **xts** → Package usefull for time-series processing
  - **tseries** → Functions for time-series analysis and modelling
  - **forecast** → Functions for time-series analysis and modelling
  - **quantmod** → Package for financial time-series analysis (includes packages **zoo, xts** and **tts**)
  - **tidyquant** → Package for financial time-series analysis (aggregates **tidyverse** and **quantmod**)
- **Basic statistical methods:**
  - **MASS** → Package extending R statistical functionalities
  - **e1071** → Package extending R statistical functionalities
  - **glmnet** → Package for applying penalized linear regression methods
  - **kernlab** → Package focused on kernel based statistical methods
- **Machine learning:**
  - **caret** → Collection of R packages for statistical modelling and machine learning
  - **mlr** → Collection of R packages for statistical modelling and machine learning
  - **mlr3** → Modernized version of the mlr package for statistical modelling and machine learning
  - **tidymodels** → **Tidyverse** based collection of packages for statistical modelling
  - **h2o** → R interface to the H2O Scalable Machine Learning Platform
  - **keras** → R interface to the Keras / Tensorflow machine learning libraries
  - **torch** → R interface to the Torch machine learning library

Some basic R packages (2)

- **Data acquisition:**
  - **readxl** → Package to import datasets from Excel (included in **tidyverse**)
  - **haven** → Allows loading of SAS, SPSS and Stata datasets (included in **tidyverse**)
  - **tidyquant** → Allows loading of data from Yahoo Finance, Quandl, FRED and other sources
  - **frenchdata** → Allows loading data from the Fama-French data library
  - **ecb** → Allows loading data from the ECB statistical data warehouse
  - **gtrendsR** → Allows loading data from Google Trends
  - **rvest** → Package for web scrapping (included in **tidyverse**)
  - **rcrawler** → More advanced package for web scrapping
- **Results reporting:**
  - **writexl** → Package for exporting data to Excel
  - **ggplot2** → Package for data plotting (included in **tidyverse**)
  - **plotly** → Package for data plotting
  - **rmarkdown** → Package to produce automated reports in HTML, Word or PDF
  - **ggvis** → Package to create ineractive charts
  - **shiny** → Package for building interactive web applications
  - **devtools** → Package for building your own R packages
- **Parallel processing:**
  - **parallel** → Package for parallel processing
  - **SNOW** → Package for parallel processing
  - **doparallel** → Package for parallel processing
  - **doSNOW** → Package for parallel processing
  - **foreach** → Package for parallel processing

Dataset processing

- Three most commonly used paradigms for **dataset processing** in R are:
  1. **base** → Standard dataset processing in base R, using the **data frame** format
  2. **tidyverse** → Simplified (and faster) dataset processing based on the **tidyverse** series of packages, using the **tibble** format (similar to data frame)
  3. **data.table** → Fastest paradigm for dataset processing (especially for large data), based on the **data.table** package, using the **data table** format (similar to data frame)
- Note: tidyverse currently includes the **dtplyr** package, which allows working with the data table format by using the tidyverse (dplyr) methodology
- For comprehensive description of the differences between the three data processing paradigms see:
  - https://mgimond.github.io/rug_2019_12/Index.html
- We will further focus on the **tidyverse** paradigm as it is the basis of the **tidyquant** package, which we will use for financial data processing

Introduction to Tidyverse

- Introduction to R packages
- **Introduction to Tidyverse**
- Introduction to Tidyquant



Introduction to Tidyverse

- **Core tidyverse packages:** (load automatically with tidyverse)
  - **readr** → Package to load text data formats (csv, tsv, fwf)
  - **tibble** → Package that adds the **tibble** data format (improved version of data frame)
  - **dplyr** → Main tidyverse package used for data processing
  - **tidyr** → Package to transform data between „wide“ and „long“ (tidy) format
  - **purrr** → Package that adds functional programming tools to tidyverse
  - **ggplot2** → Package for data plotting
  - **stringr** → Package for working with strings
  - **forcats** → Package for working with factor variables
- **Additional tidyverse packages**: (need to be loaded separatelly)
  - **haven** → Package to load SAS, SPSS and Stata data
  - **readxl** → Package to read Excel data
  - **rvest** → Package for web data scrapping
  - **lubridate** → Package for working with dates and times
  - **dbplyr** → Package for working with SQL databases
  - **dtplyr** → Package for working with data tables
  - **magittr** → Package that introduced the pipe operator and its extensions
  - **glue** → Package for combining data and strings
  - (+ many others)

Some tidyverse basics

❖ Tibble

- Data structure that is very similar to base R **data frame**
- Used to store datasets where each column contains values of a single data type
- When subsetting a tibble with [], the result will also be a tibble
- When printing a tibble it prints only the first 10 rows
- Unlike data frames, tibbles do not particularly support row names

❖ Wide vs. long (tidy) data format

Wide format:

Month	Stock_A	Stock_B	Stock_C
Jan-21	5%	3%	4%
Feb-21	-5%	-3%	-4%
Mar-21	10%	6%	8%

Long (tidy) format

Month	Stock	Return
Jan-21	A	5%
Feb-21	A	-5%
Mar-21	A	10%
Jan-21	B	3%
Feb-21	B	-3%
Mar-21	B	6%
Jan-21	C	4%
Feb-21	C	-4%
Mar-21	C	8%

- Tidyverse functions generally prefer the long (tidy) dataset format (typical in databases)
- Conversion between the wide and long format can be performer with functions in the tidyr package

❖ Pipe operator

- **Pipe operator:** `X %>% FUNCTION`
- Allows us to repeatedly apply dataset modifications to dataset X
- Simplifies writing of complex data transformations with the dplyr package

Loading data with tidyverse

- **Package readr:**
  - **Read csv data:** `z <- read_csv('NAME.csv')`
  - **Read tsv data:** `z <- read_tsv('NAME.tsv')`
  - **Read delimited data:** `z <- read_delim('NAME.txt', delim = '|')`
- **Package haven:**
  - **Read SAS data:** `z <- read_sas('NAME.sas7dbat')`
  - **Read SPSS data:** `z <- read_spss('NAME.sav')`
  - **Read Stata data:** `z <- read_dta('NAME.dta')`
- **Package readxl:**
  - **Read xls data:** `z <- read_xls('NAME.xls')`
  - **Read xlsx data:** `z <- read_xlsx('NAME.xlsx')`
  - **Read Excel data:** `z <- read_excel('NAME.xlsx‘)` (xls, xlsx, xlsm, etc.)
- **Package tibble:**
  - **Create a tibble:** `z <- tibble ('x'=x, 'y'=y, 'z'=y)`
  - **Convert to tibble format:** `z <- as_tibble(x)`

Tibble format - Illustration

- Let us start by loading the **tidyverse** package:

```
# Load tidyverse
library(tidyverse)
```

- Load the default R dataset **mtcars** as **data frame:**

```
# mtcars dataset (data.frame format)
mtcars_orig <- mtcars
class(mtcars_orig)
[1] "data.frame"
```

- Load the default R dataset **mtcars** as **tibble:**

```
# Load mtcars (tibble format)
mtcars_tbl <- as_tibble(mtcars)
class(mtcars_tbl)
[1] "tbl_df"     "tbl"        "data.frame"
```

- We can then use standard functions to explore the dataset, such as:
  - **nrow:** `nrow(mtcars_tbl)` (number of rows in the dataset
  - **head:** `head(mtcars_tbl,5)` (show first 5 rows)
  - **tail:** `tail(mtcars_tbl,5)` (show last 5 rows)
  - **slice:** `slice(mtcars_tbl,3:6)` (show rows 3 to 6)
  - **glimpse:** `glimpse(mtcars_tbl)` (show variable names and types)
  - **summary:** `summary(mtcars_tbl)` (summary statistics of the dataset)

Data processing with dplyr

- **Package dplyr:**
  - **Filter rows:** `filter(X, CONDITION)`
  - **Sort rows:** `arrange(X, COLUMN)`
  - **Select columns:** `select(X, COLUMNS)`
  - **Transform column:** `mutate(X, COLUMN=FUNCTION)`
  - **Group by variable:** `group_by(X, COLUMN)`
  - **Summarise:** `summarise(X, y=FORMULA)`
  - **Transform columns:** `mutate_at(X,.vars=(COLUMNS), .funs=(FUNCTION))`
  - **Summarise columns:** `summarise_at(X,.vars=(COLUMNS), .funs=(FUNCTION))`
- **Pipe operator**
  - **Pipe operator:** `X %>% FUNCTION`

Filter function

- Standard call of the **filter function**:
  - Filter **single value** based on **single variable:**

```
# Filter function
filter(mtcars_tbl,cyl==8)
```

  - Filter **multiple values** based on **single variable:**

```
# Filter multiple values
filter(mtcars,cyl %in% c(4,6))
```

  - Filter based on **multiple variables:**

```
# Filter based on multiple variables
filter(mtcars_tbl,(cyl %in% c(4,6)) & (carb %in% c(1,2)))
```

  - Filter using a **not statement:**

```
# Filter using a not statement
filter(mtcars_tbl,cyl != 4)
```

- The same filters can also be applied by using the **pipe operator**:

```
# Filter using multiple variables
mtcars_tbl %>%
  filter((cyl %in% c(4,6)) & (carb %in% c(1,2)))
```

- Which allows us to **stack multiple filters** (and other transformations):

```
# Two stacked filters
mtcars_tbl %>%
  filter(cyl %in% c(4,6)) %>%
  filter(carb %in% c(1,2))
```

Arrange function

- Standard call of the **arrange function**:

```
# Sort the dataset using the arrange function
arrange(mtcars_tbl,mpg)
```

- The same can be performer also with the **pipe operator**:

```
# Sort using arrange and the pipe operator
mtcars_tbl %>%
  arrange(mpg)
```

- We can also sort the tibble in **descending order**:

```
# Sort in descending order
mtcars_tbl %>%
  arrange(desc(mpg))
```

- Or sort it based on **multiple variables**:

```
# Sort based on multiple variables
mtcars_tbl %>%
  arrange(desc(cyl),desc(hp))
```

- To **save the sorted result** into a new variable, use:

```
# Sort based on multiple variables - save the output
mtcars_s <- mtcars_tbl %>%
  arrange(desc(cyl),desc(hp))
print(mtcars_s)
```

Select function

- Standard call of the **select function** to select some of the variables:

```
# Select function
select(mtcars_tbl,cyl,mpg)
```

- The same can be performer also with the **pipe operator**:

```
# Select using the pipe operator
mtcars_tbl %>%
  select(cyl,mpg)
```

- We can also use select to **drop some variables**:

```
# Drop some variable
mtcars_tbl %>%
  select(-mpg)
```

- Or **select solumns** based on a **condition**:

```
# Select columns based on condition
x <- mtcars_tbl %>%
  select(starts_with("c"),starts_with("h"))
```

Mutate function

- We can use the **mutate function** to change the variable type:

```
# Convert numeric variable into factor
mtcars2 <- mtcars_tbl %>%
  mutate(cyl = factor(cyl,
                      levels = c(4,6,8),
                      labels = c("4 Cyl","6 Cyl","8 Cyl")))
```

- Or to **create new variables**:

```
# Convert miles to liters
mtcars2 <- mtcars_tbl %>%
  mutate(miles_per_liter = mpg * 3.78541,
         miles_per_galon = miles_per_liter / 3.78541)
```

- If we want the new dataset to contain only the transformed variables, we can use the **transmute function** instead:

```
# Convert miles to liters + select only the transformed variables
mtcars2 <- mtcars_tbl %>%
  transmute(miles_per_liter = mpg * 3.78541,
         miles_per_galon = miles_per_liter / 3.78541)
```

Group by function

- Assign **groups** and take the **first observation** of each group:

```
# Assing groups and take the first observations
mtcars_tbl %>%
  group_by(cyl) %>%
   slice(1)
```

- Assign **groups** and take the **maximum of some variable** from each group:

```
# Filter by group to find the maximum of each group
mtcars_tbl %>%
  group_by(cyl) %>%
    filter(hp == max(hp))
```

- Or the **n largest observations** from each group:

```
# Find the maximum of the groups using arrange and slice
x <- mtcars_tbl %>%
  group_by(cyl) %>%
    arrange(desc(hp)) %>%
      slice(3)
```

- Group by **multiple** variables and compute **counts of observations in each group**:

```
# Group by two variables and compute the group counts
x <- mtcars_tbl %>%
  group_by(cyl,am) %>%
    count()
```

Summarize function

- Compute the **mean** and **median** of a **single variable**:

```
# Compute the mean and median of mpg
mtcars_tbl %>%
  summarize(mean = mean(mpg),
            median = median(mpg))
```

- Compute the **mean**, **standard deviation** and **count** for different **groups**:

```
# Compute mean, standard deviation and count for different groups
mtcars_tbl %>%
  group_by(cyl) %>%
    summarize(mean_mpg = mean(mpg),
              sd_mpg = sd(mpg),
              n = n())
```

mutate_at & summarize_at

- Modify **multiple variables** at once:

```
# Mutate multiple variables
mtcars_tbl %>%
  mutate_at(.vars = vars(cyl,am),factor) %>%
    select(cyl,am) %>%
      glimpse()
```

- Summarize **multiple variables** at once:

```
# Means of multiple variables
mtcars_tbl %>%
  group_by(cyl) %>%
   summarize_at(.vars = vars(mpg,disp,qsec),.funs = mean)
```

- Apply **multiple summary functions** to **multiple variables** at once:

```
# Compute means and standard deviations of multiple variables
mtcars_tbl %>%
  group_by(cyl) %>%
  summarize_at(.vars = vars(mpg,disp,qsec),.funs = funs(mean,sd))
```

Data transformations with tidyr

- **Package dplyr:**
  - **Gather columns into long format:** `gather(NAME, VALUES, NAME_1:NAME_N)`
  - **Spread columns into wide format:** `spread(NAME, VALUES)`
  - **Separate one column into more:** `separate("NAME", into=c("NAME1","NAME2"))`
  - **Unite multiple columns into one:** `unite("NAME", c("NAME1","NAME2"))`

Functions gather and spread

- We will start by creating a **tibble** of stock returns in the **untidy (wide)** format:

```
# Create a tibble of stock returns
untidy_df <- tibble("year" = c(2018,2019,2020),
                    "A" = c(0.1,-0.05,0.15),
                    "B" = c(0.05,-0.1,0.1),
                    "C" = c(0.2,-0.2,0.3))
```

- We can then convert it into the **tidy (long)** format with **gather**:

```
# Convert to tidy (long) format
tidy_df <- untidy_df %>%
  gather(stock,return,A:C)
```

- And we can convert it back into the **untidy (wide)** format with **spread**:

```
# Convert to untidy (wide) format
untidy_df_2 <- tidy_df %>%
  spread(stock,return)
```

- When in tidy format, we can also **unite** the two columns:

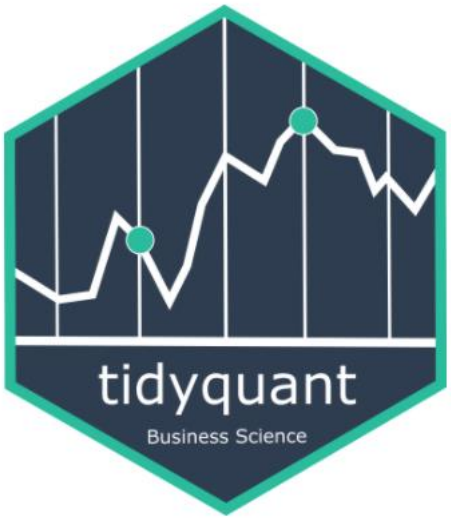
```
# Join columns with unite
tidy_df <- tidy_df %>%
  unite("stock_year",c("stock","year"),sep = "_")
```

- Or **separate** the united columns:

```
# Separate columns with separate
tidy_df <- tidy_df %>%
  separate(stock_year, into = c("stock","year"),sep = "_")
```

Introduction to Tidyquant

- ✓ Introduction to R packages
- ✓ Introduction to Tidyverse
- ➢ **Introduction to Tidyquant**
  - ➢ **Basic Tidyquant functions**
  - ❖ Processing stock price data
  - ❖ Case study 1 – S&P500 crashes
  - ❖ Processing data for multiple stocks
  - ❖ Case study 2 – Factor investing



Introduction to Tidyquant

- **Tidyquant** unites the logic of:
  - **tidyverse** → Collection of packages for data processing, cleaning and visualization
- With functions contained in specialized **time-series** and **financial** packages:
  - **quantmod** → Package for financial data loading, processing and transformation
  - **zoo** → Package for time-series processing and analysis
  - **xts** → Package for time-series processing and analysis
  - **tts** → Package for technical analysis implementation
  - **PerformanceAnalytics** → Package for investment analysis and performance measurement
  - (+) Tidyquant implements also many popular MS Excel functions into R
- **Core tidyquant functions**:
  - **Get financial data:** `tq_get(SYMBOL,get=WHAT,from=DATE1,to=DATE2)`
  - **Transform column:** `tq_mutate(DATA,X,FUNCTION)` (output in DATA)
  - **Transform column:** `tq_transmute(DATA,X,FUNCTION)` (output in new tibble)
  - **Compute performance:** `tq_performance(DATA,Ra,Rb,FUNCTION)`
  - **Create portfolio:** `tq_portfolio(DATA,ASSETS,RETURNS,WEIGHTS)`
- Functions **tq_mutate** and **tq_mutate** mimic the tidyverse functions mutate and transmute but allow for the use of **time-series** transformation functions from the **zoo, xts, tts** and **PerformanceAnalytic** packages

Tidyquant – Load financial data

- We can use function **tq_get** to automatically download online financial data into R (in a Tibble format) from multiple data sources (Yahoo finance, FRED, Quandl, etc.)
- To **load financial data** we can call the function as follows:

```
tq_get(SYMBOL,get=SOURCE,from=DATE1,to=DATE2)
```

- **SYMBOL** denotes the **list of symbols** that we want to get, **SOURCE** denotes the data source, **DATE1** the starting date of the time series and **DATE2** the ending date
- To get a list of eligible data sources, we can use the function **tq_get_options()**:

```
tq_get(SYMBOL,get=SOURCE,from=DATE1,to=DATE2)
```

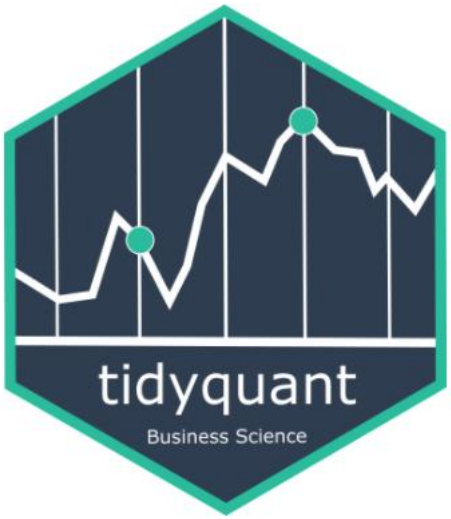
- Which gives the following output:

```
[1] "stock.prices"       "stock.prices.japan"
[3] "dividends"          "splits"
[5] "economic.data"      "quandl"
[7] "quandl.datatable"   "tiingo"
[9] "tiingo.iex"         "tiingo.crypto"
[11] "alphavantager"      "alphavantage"
[13] "rblpapi"
```

- Among the sources / data types that we will use the most are:
  - **stock.prices** = OHLCV stock prices from Yahoo Finance
  - **dividends** = Dividend data from Yahoo Finance
  - **splits** = Stock split data from Yahoo finance
  - **economic.data** = Economic data from the FRED database

Introduction to Tidyquant

- ✓ Introduction to R packages
- ✓ Introduction to Tidyverse
- ➢ **Introduction to Tidyquant**
  - ✓ Basic Tidyquant functions
  - ➢ **Processing stock price data**
  - ❖ Case study 1 – S&P500 crashes
  - ❖ Processing data for multiple stocks
  - ❖ Case study 2 – Factor investing



Load OHLCV data for AAPL

- **Task:** download daily OHLCV price data for Apple (symbol AAPL) in the period since 1.1.2001 until today, we can run the code
- We **load the tidaquant** package**:**

```
# Load tidyquant
library(tidyquant)
```

- Define the **settings** of the data request:

```
# Settings:
symbol <- "AAPL"
tStart <- "2001-01-01"
tEnd   <- today()
```

- And then load the data with **tq_get**:

```
# Load stock prices
prices <- tq_get(symbol, get = "stock.prices", from = tStart, to = tEnd)
```

- We can then check the contents of the result (**prices**) by calling:

```
# Show the result
head(prices)
```

- Or by simply opening it in the RStudio

OHLCV data for AAPL

- RStudio result of AAPL data load with tq_get:

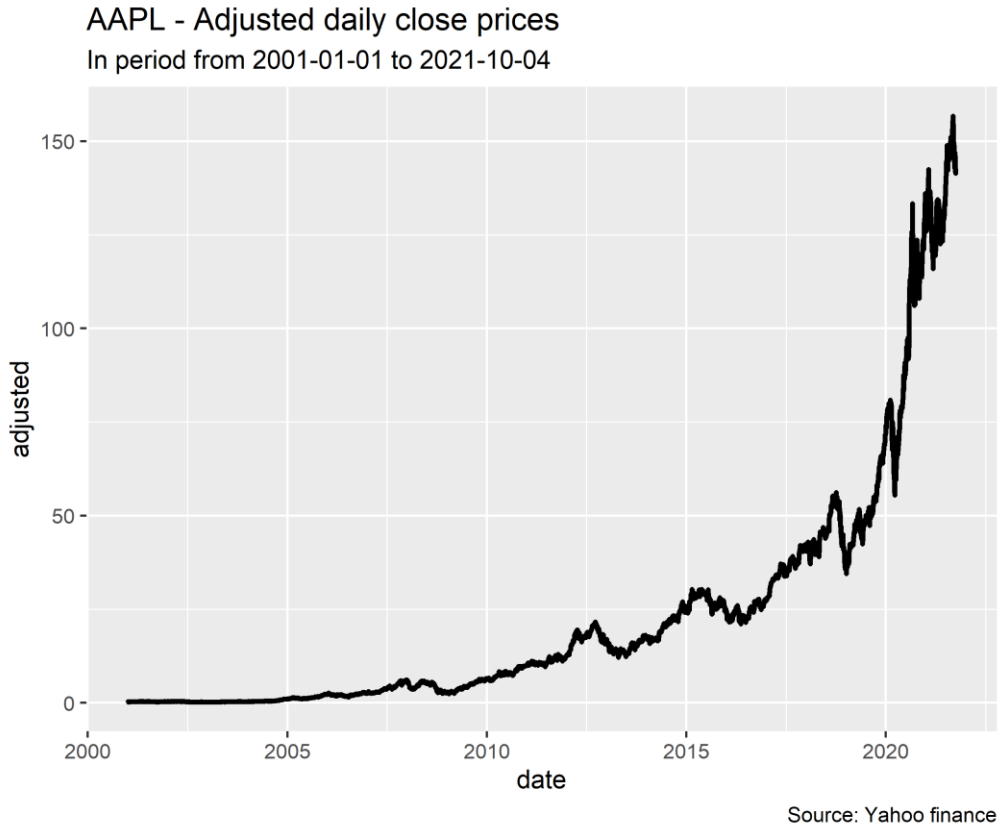
	symbol	date	open	high	low	close	volume	adjusted
1	AAPL	2001-01-02	0.265625	0.272321	0.260045	0.265625	452312000	0.228070
2	AAPL	2001-01-03	0.258929	0.297991	0.257813	0.292411	817073600	0.251069
3	AAPL	2001-01-04	0.323940	0.330357	0.300223	0.304688	739396000	0.261610
4	AAPL	2001-01-05	0.302455	0.310268	0.286830	0.292411	412356000	0.251069
5	AAPL	2001-01-08	0.302455	0.303292	0.284598	0.295759	373699200	0.253944

- Where:
  - **symbol** = Symbol of the loaded stock
  - **date** = Date of the daily prices
  - **open** = Opening price of the day (stock-split adjusted)
  - **high** = Highest price of the day (stock-split adjusted)
  - **low** = Lowest price of the day (stock-split adjusted)
  - **close** = Closing price of the day (stock-split adjusted)
  - **volume** = Trading volume in number of stocks (stock-split adjusted)
  - **adjusted** = Closing price of the day (stock-split & dividend adjusted)
- Note: Yahoo Finance does not report the unadjusted stock prices, the **OHLCV** prices are all **stock-split adjusted**, while the **adjusted** price is in addition to that also **dividend-adjusted**

AAPL – Stock price evolution (1)

- We can draw the **adjusted close price evolution** as follows:

```
# Plot the evolution of adjusted close prices
prices %>%
  ggplot(aes(x = date, y = adjusted)) +
  geom_line(size = 1) +
  labs(title = paste0(symbol, " - Adjusted daily close prices"),
       subtitle = paste0("In period from ",tStart," to ",tEnd),
       caption = "Source: Yahoo finance")
```

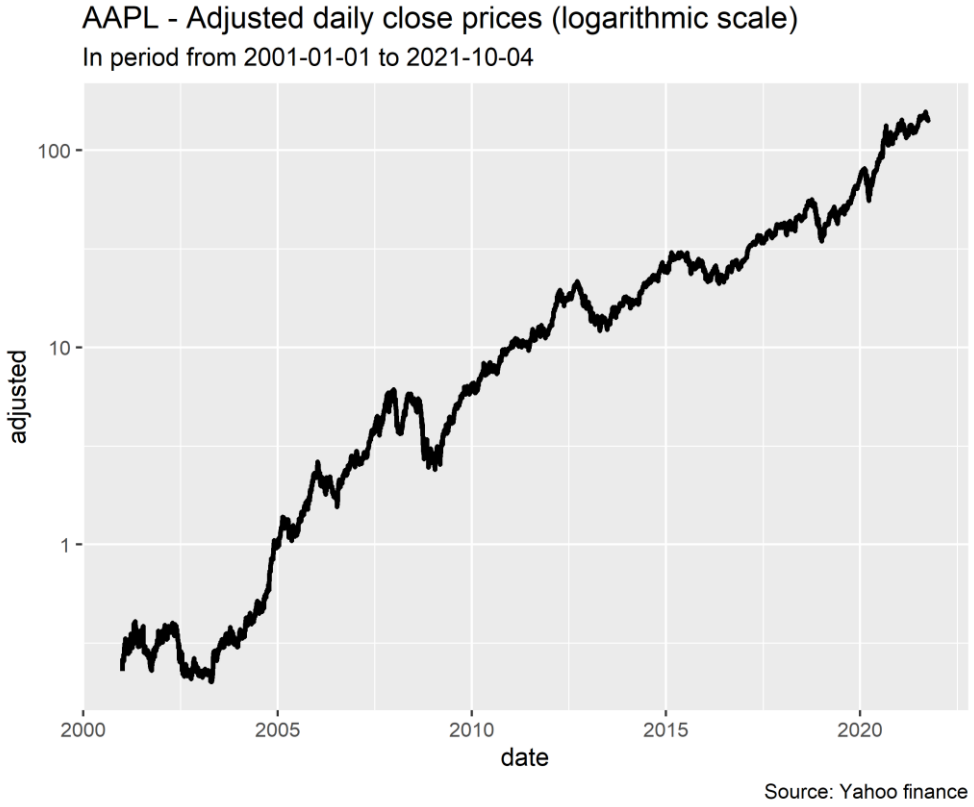


- The **adjusted close prices** (for stock-splits and dividends) show how the value of an investment into AAPL would have evolved in the period since 1.1.2001
- We can observe **exponential growth**, typical for stocks
- We can **linearize** the stock price behavior, by computing a **logarithm of the stock price** or plotting the prices on a chart with a **logarithmic scale**

AAPL – Stock price evolution (2)

- We can draw the **adjusted close price evolution** on a **logarithmic scale** as follows:

```
# Plot the evolution of  adjusted close prices - Log scale
prices %>%
  ggplot(aes(x = date, y = adjusted)) +
  geom_line(size = 1) +
  scale_y_log10() +
  labs(title = paste0(symbol, " - Adjusted daily close prices (log-scale)"),
       subtitle = paste0("In period from ",tStart," to ",tEnd),
       caption = "Source: Yahoo finance")
```

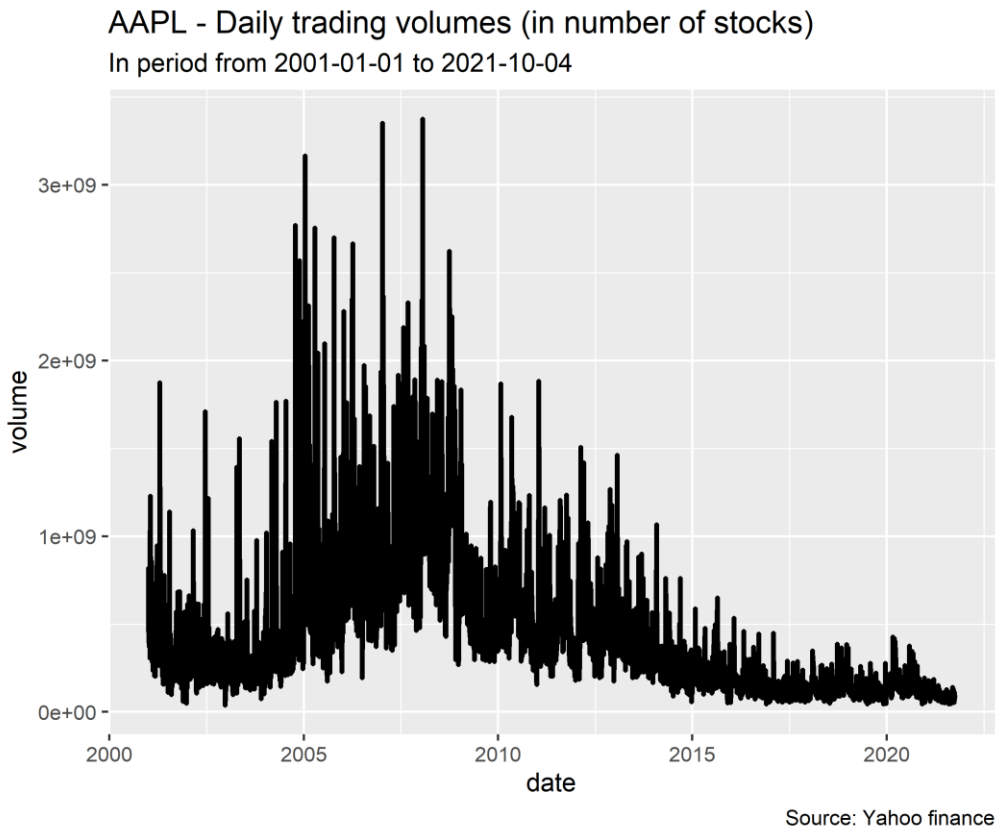


- Exponential growth on a standard scale corresponds to a linear growth on a logarithmic scale
- We can see that the AAPL stock price was growing more or less exponentially during the period since 1.1.2000 (i.e. on a logarithmic scale it is growing linearly)

AAPL – Volume evolution (1)

- We can also plot the **evolution of daily trading volume** (in number of stocks):

```
# Plot the evolution of trading volumes (in number of stocks)
prices %>%
  ggplot(aes(x = date, y = volume)) +
  geom_line(size = 1) +
  labs(title = paste0(symbol, " - Daily trading volumes (in number of stocks)"),
       subtitle = paste0("In period from ",tStart," to ",tEnd),
       caption = "Source: Yahoo finance")
```

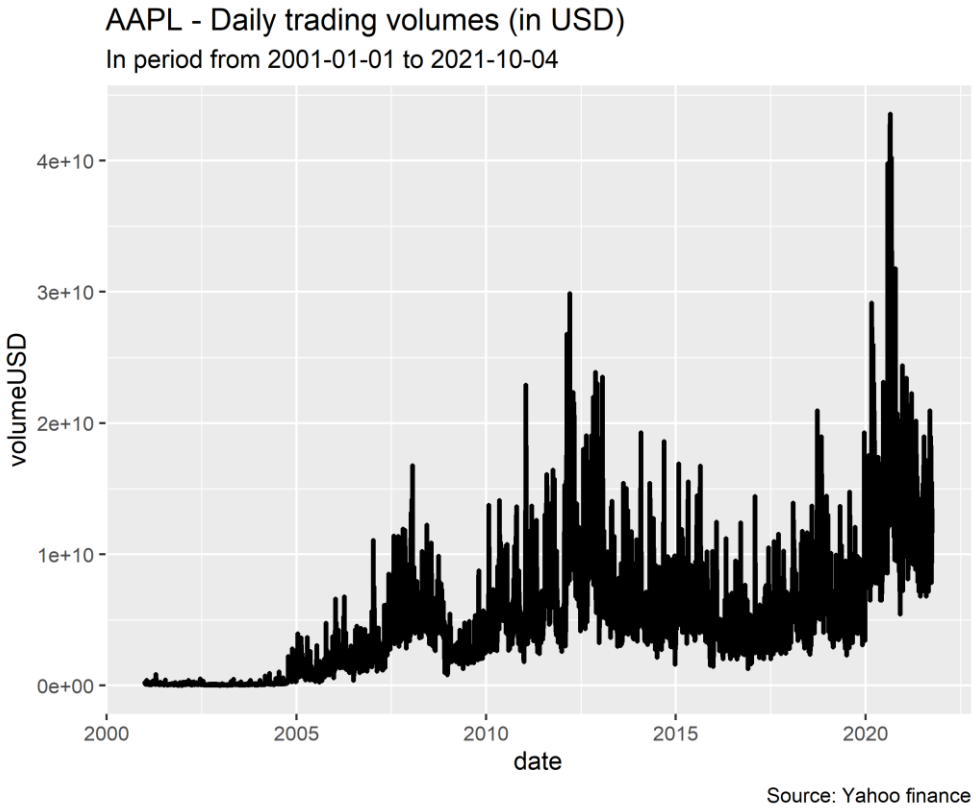


- **Note:** Similarly as the stock prices, Yahoo Finance **adjusts the trading volume for stock splits (!)**
- The evolution of the trading volume in number of stocks is thus **not very representative**
- To get a representative characteristics of trading volumes, we need to convert them into **USD volumes** (by multiplying them with the stock price)

AAPL – Volume evolution (2)

- We can also plot the **evolution of daily trading volume** (in number of stocks):

```
# Plot the evolution of trading volumes (in USD)
prices %>%
  mutate(volumeUSD = volume*close) %>%
  ggplot(aes(x = date, y = volumeUSD)) +
  geom_line(size = 1) +
  labs(title = paste0(symbol, " - Daily trading volumes (in USD)"),
       subtitle = paste0("In period from ",tStart," to ",tEnd),
       caption = "Source: Yahoo finance")
```

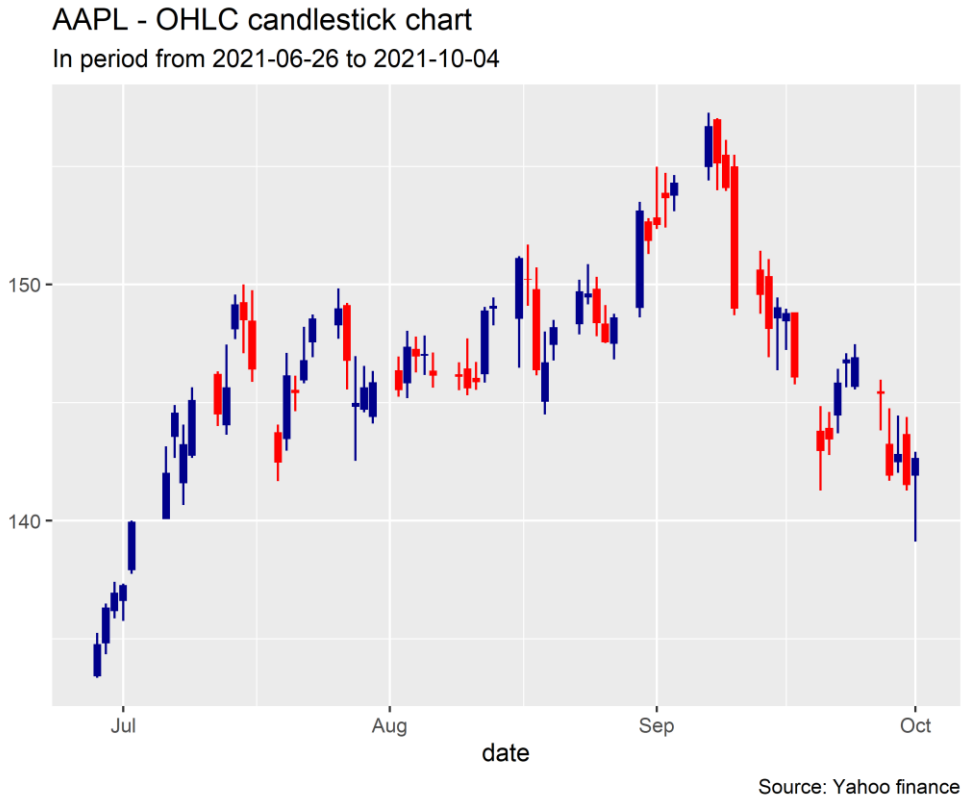


- We can see that the USD trading volume of AAPL is actually increasing (as expected)
- We can see that the daily trading volume for AAPL tends to be around 10-40 billion USD per day the last few years (it is one of most traded stocks in the US)

AAPL – Candlestick chart

- We can also plot the OHLC prices of AAPL on a **candlestick chart**:

```
# Plot OHLC candlestick stock prices (last 100 days)
c06 <- prices %>%
  filter(date >= tEnd-100) %>%
  ggplot(aes(x = date)) +
  geom_candlestick(aes(open = open, high = high, low = low, close = close)) +
  labs(title = paste0(symbol, " - OHLC candlestick chart"),
       subtitle = paste0("In period from ",tEnd-100," to ",tEnd),
       caption = "Source: Yahoo finance")
```



- **Candlestick chart** shows the **open, high, low** and **close** prices of the stock on a single chart
- The chart is popular tool in technical analysis, as it allows us to see support/resistence levels and candlestic patterns
- For example on the last day of the sample (30.10.2021), the price has formed the **pinbar / hammer** pattern, which is typically a **bullish pattern** in technical analysis

Dividend & stock-split data (AAPL)

- **Task:** In addition to the OHLCV price data for Apple (symbol AAPL) in the period since 1.1.2000, download also the **dividend** and **stock-split** data

- Define the **settings** of the data request:

```
# Settings:
symbol <- "AAPL"
tStart <- "2001-01-01"
tEnd   <- today()
```

- Download the **OHLCV** data:

```
# Load stock prices
prices <- tq_get(symbol, get = "stock.prices", from = tStart, to = tEnd)
```

- Download the **dividend** data:

```
# Load dividends
dividends <- tq_get(symbol, get = "dividends", from = tStart, to = tEnd)
```

- Download the **stock split** data:

```
# Load stock splits
splits <- tq_get(symbol, get = "splits", from = tStart, to = tEnd)
```

Dividends & Stock-Splits (AAPL)

❖ **Dividend data for AAPL:**

	symbol	date	value
1	AAPL	2012-08-09	0.003380107
2	AAPL	2012-11-07	0.003380107
3	AAPL	2013-02-07	0.003380107
4	AAPL	2013-05-09	0.003890321
5	AAPL	2013-08-08	0.003890321

Stock-Split data for AAPL:

	symbol	date	value
1	AAPL	2005-02-28	0.5000000
2	AAPL	2014-06-09	0.1428571
3	AAPL	2020-08-31	0.2500000

❖ The **dividend dataset** contains:

- **symbol** = Symbol of the loaded stock
- **date** = Ex-dividend date of the dividend (**not** the announcement or payment date!)
- **value** = Dividend per stock in USD (**not** stock-split adjusted!)

❖ The **stock-split dataset** contains:

- **symbol** = Symbol of the loaded stock
- **date** = Date of the stock split
- **value** = Ratio used in the stock split

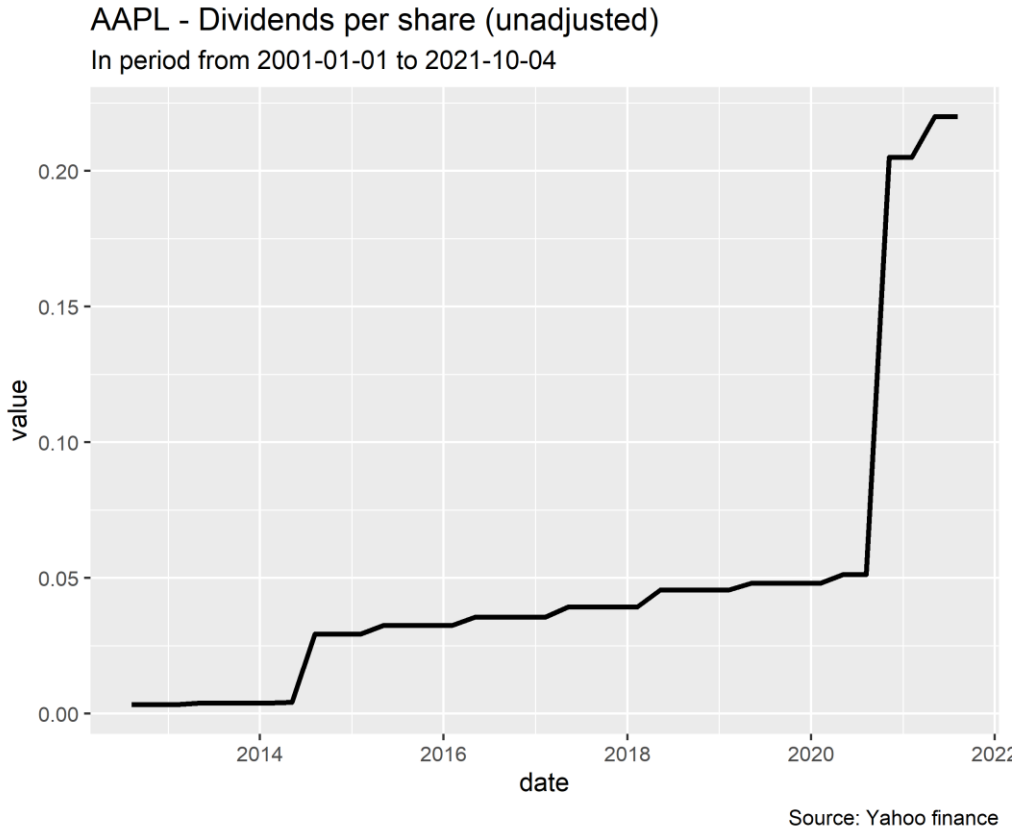
❖ We can see that AAPL performed 3 stock-splits since 1.1.2000:

1. **2005-02-28 → 1:2 stock split →** Number of stocks increases **2x** & stock price drops to **0.5**
2. **2014-06-09 → 1:7 stock split →** Number of stocks increases **7x** & stock price drops to **0.1428571**
3. **2020-08-31 → 1:4 stock split →** Number of stocks increases **4x** & stock price drops to **0.25**

AAPL – Dividend evolution

- We can plot the raw **dividend** data (unadjusted for stock-splits) as follows:

```
# Plot dividend data (unadjusted)
dividends %>%
  ggplot(aes(x = date, y = value)) +
  geom_line(size = 1) +
  labs(title = paste0(symbol, " - Dividends per share (unadjusted)"),
       subtitle = paste0("In period from ",tStart," to ",tEnd),
       caption = "Source: Yahoo finance")
```



- The **unadjusted** dividend-per-share data are, however, **not representative** as they are influenced by the changes in the number of shares issued
- We can see **two major spikes** in the dividend behavior, corresponding to the:
  1. **2014-06-09 stock split**
  2. **2020-08-31 stock split**
- Take note that we do not see the effect of the **2005-02-28 stock split** as AAPL was not paying dividends before 2013

Stock-split adjusted dividends

- In order to get a more representative dividend measure, we can **adjust the dividends** for the previous stock-splits (in a similar way as Yahoo Finance performs the adjustment of the closing prices), by **dividing them with split ratios**
- As tidyverse (unfortunatelly) does not allow for inequality joins, we will perform the adjustment with a cycle:

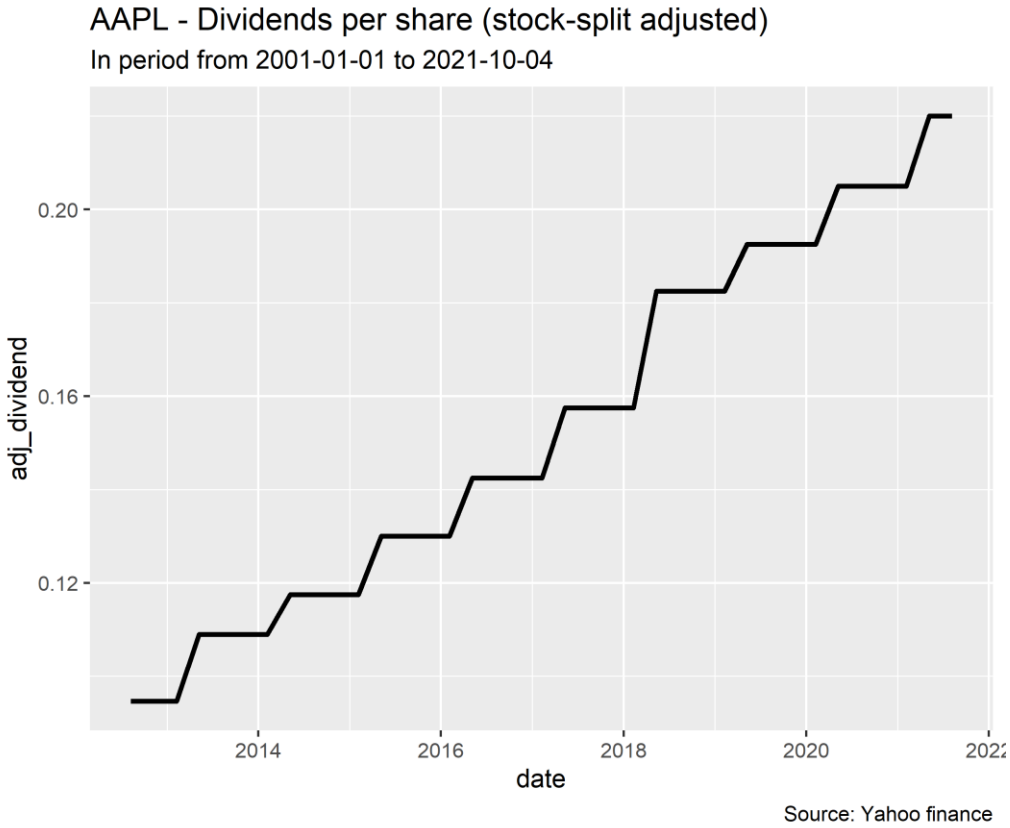
```
# Computing stock-split adjusted dividends
kSplits <- nrow(splits)
dividends$adj_dividend <- dividends$value
for (i in (1:kSplits)) {
  splitDate <- splits$date[i]
  splitValue <- splits$value[i]
  dividends <- dividends %>%
    mutate(adj_dividend =
    ifelse(date<=splitDate,adj_dividend/splitValue,adj_dividend))
  }
```

- The code above cycles over the stock-split dates, and divides the dividends before each of the split dates by the stock-split ratios occuring after that split date
- The transformations will tell us the historical values of **adjusted dividends per share** that **correspond to the current number of stocks** (after all of the stock splits) and are thus comparable over time

AAPL – Adjusted dividends

- We can plot the **adjusted dividend** data (i.e. adjusted for stock-splits) as follows:

```
# Plot dividend data - Adjusted for stock splits
dividends %>%
  ggplot(aes(x = date, y = adj_dividend)) +
  geom_line(size = 1) +
  labs(title = paste0(symbol, " - Dividends per share (stock-split adjusted)"),
       subtitle = paste0("In period from ",tStart," to ",tEnd),
       caption = "Source: Yahoo finance")
```



- We can see that **AAPL** has been systematically increasing its dividend-per-share (transformed to current number of shares) over the analyzed time period
- While usefull for the analysis of the trend, the chart is **not so usefull for stock comparison**, as we do not know how much did the investors need to pay in order to get these dividends
- For inter-stock comparibility, we will compute the **dividend yield**

Dividend yield calculation

- To get a simple estimate of the **dividend yield**, we will divide the **stock-split adjusted dividend per share** with the **stock-split adjusted price per share** at the time of the dividend payment, and multiply the result by 4 to annualize the estimate (as the dividends are paid quarterly)
- To do that, we need to **join the close price data to the dividend data** and proceed as follows:

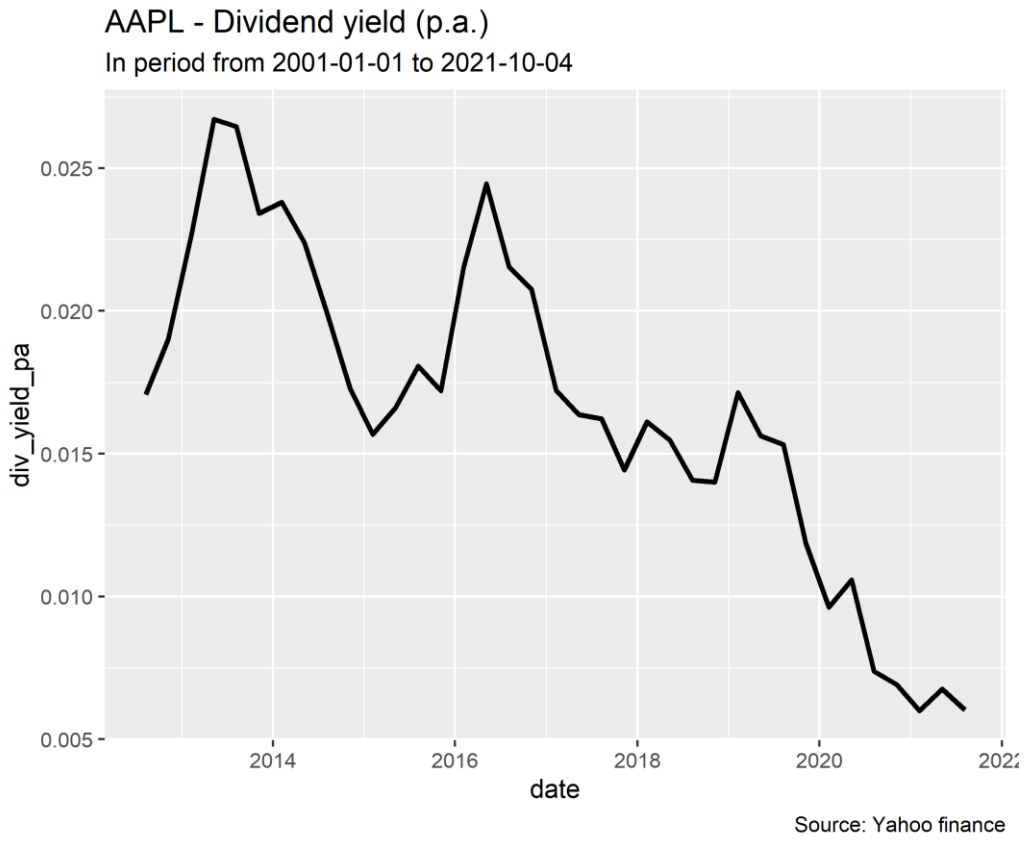
```
# Calculate dividend yield
temp <- prices %>%
  select(date, close)
dividends2 <- dividends %>%
  left_join(temp,by="date") %>%
  mutate(div_yield_pa = (adj_dividend / close)*4)
```

- The **div_yield_pa** variable then tells us an estimate of the dividend yield (computed from the dividend per share and the price per share on the same date)

AAPL – Dividend yield

- We can plot the **adjusted dividend** data (i.e. adjusted for stock-splits) as follows:

```
# Plot dividend data - Adjusted for stock splits
dividends %>%
  ggplot(aes(x = date, y = adj_dividend)) +
  geom_line(size = 1) +
  labs(title = paste0(symbol, " - Dividends per share (stock-split adjusted)"),
       subtitle = paste0("In period from ",tStart," to ",tEnd),
       caption = "Source: Yahoo finance")
```



- We can see that inspite of the **dividends per share** increasing over the analyzed time period, the **dividend yield** has been **decreasing** as the AAPL stock price has been growing even faster than the dividends
- While in 2013-2017, the dividend yield of AAPL oscillated around 2% p.a., currently it has dropped to only 0.6% p.a.
- Unlike dividends per share, the dividend yield can be used to compare stocks with each other

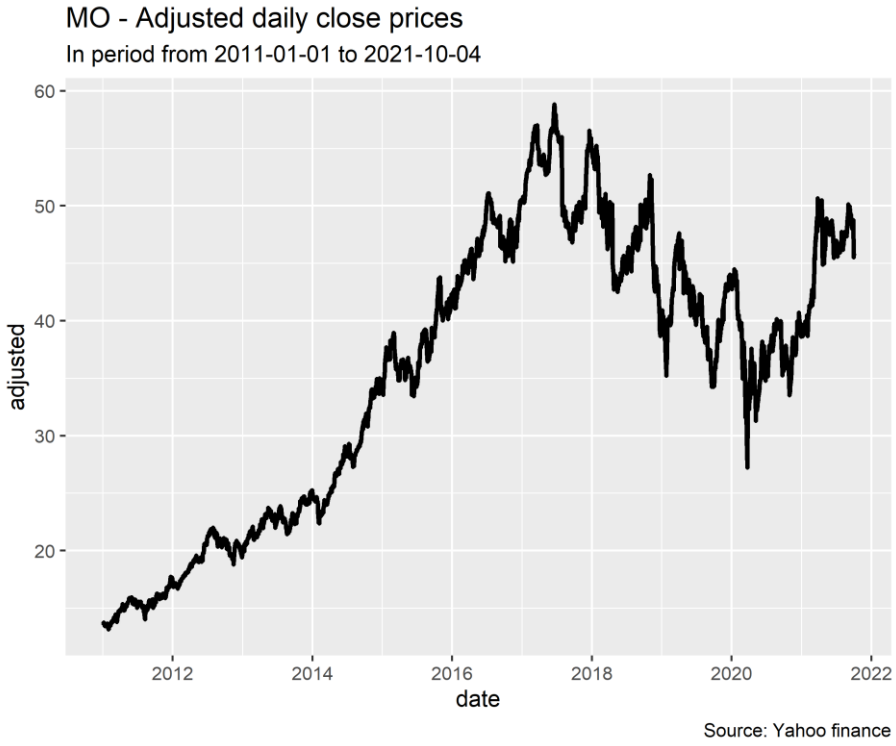
Altria stock price & dividends

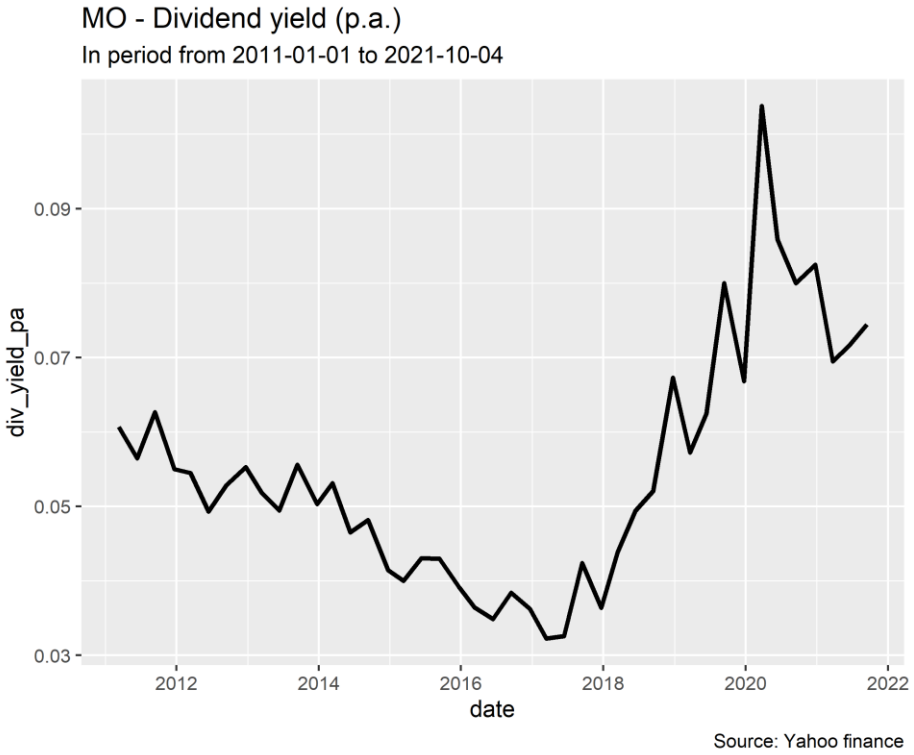
- **Task:** Repeat the previous analyses for Altria (symbol MO) in the period since 1.1.2011 until today (**Note:** We have shortened the time period due to some extraordinary dividends paid by MO in the previous years that bias the charts)
- To repeat the analyses, we simply need to change the **settings** as follows:

```
# Settings:
symbol <- "MO"
tStart <- "2011-01-01"
tEnd   <- today()
```

- And re-run all of the codes

Altria – Stock price & Dividend yield





- We can see that in addition to growing in the period sinc 2011-01-01, the Altria stock has also been paying its holders a high **dividend yield of 3-10% p.a.** during the analyzed time period
- At its current price, the Altria stock provides a dividend yield of **7% p.a.**, which means that as long as the value of the stock does not decrease in the future, the current investors may earn 7% p.a. only from the dividends
- As the Altria stock pays relatively high dividends, it is also usefull to **illustrate why it is necessary to use dividend-adjusted stock prices** when assessing the **stock performance**

Ex-dividend price drops - Illustration

- To illustrate what happens with the close price of a stock (unadjusted for dividends) on the ex-dividend date, we will join the dividend data to the stock price data:

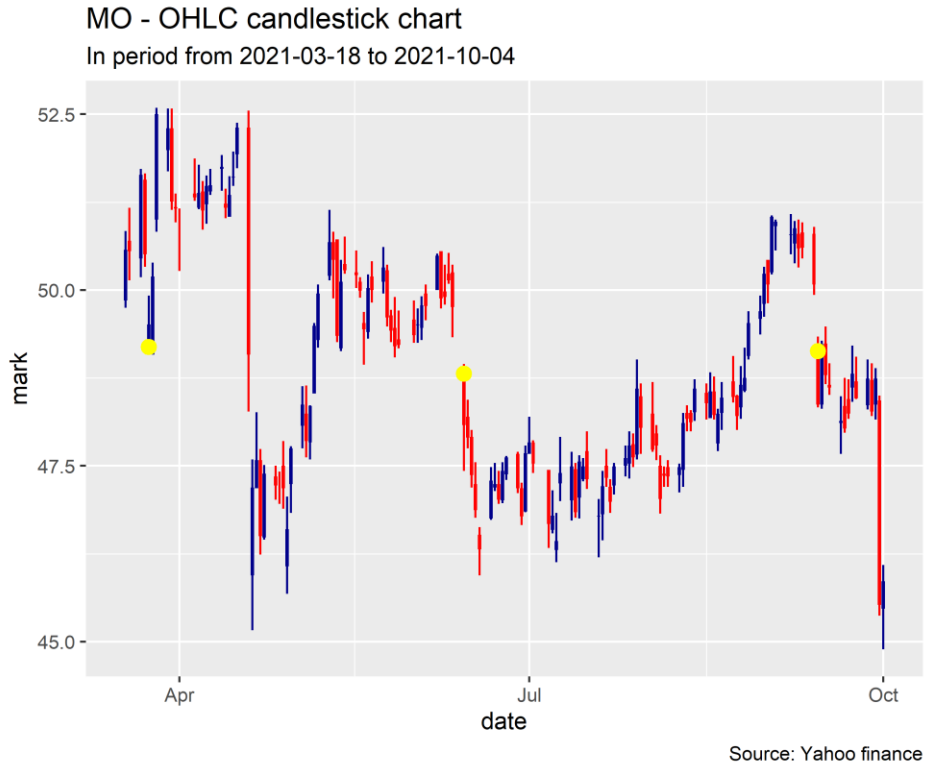
```
# Joining of the dividend and stock price data
temp <- dividends %>%
  select(date, adj_dividend)
prices2 <- prices %>%
  left_join(temp, by="date") %>%
  mutate(mark = open*(adj_dividend>0))
```

- Where the newly created variable **mark** will be used to mark the **opening price** of the stock on the **ex-dividend date**

MO – Candlestick chart

- We can also plot the OHLC prices of AAPL on a **candlestick chart**:

```
# Plot OHLC candlestick stock prices (last 200 days)
prices2 %>%
  filter(date >= tEnd-200) %>%
  ggplot(aes(x = date)) +
  geom_candlestick(aes(open = open, high = high, low = low, close = close)) +
  geom_point(aes(y = mark),size = 3, color="yellow") +
  labs(title = paste0(symbol, " - OHLC candlestick chart"),
       subtitle = paste0("In period from ",tEnd-200," to ",tEnd),
       caption = "Source: Yahoo finance")
```



- We can see that the **opening price** on the **ex-dividend** date (**yellow dots**) always drops compared to the previous-day close price
- This is because the firm has paid part of its value to the shareholders in the form of the dividend
- These drops in price do, however, not result in any loss to the stock holders, as whatever they lost from the price drop they received in the form of dividends
- In order to measure stock performance, we thus always need to use **dividend-adjusted stock prices**

Dividend adjusted OHLC prices

- To compute adjusted OHLC prices, we need to adjust the prices before each of the ex-dividend dates by a factor corresponding to 1 minus the dividend yield (p.q.)
- As this has already been done by Yahoo for the closing price, we can extract the adjustment factors from the ratio between the dividend-adjusted and dividend-unadjusted close prices and use it to adjust the open, high and low prices as well

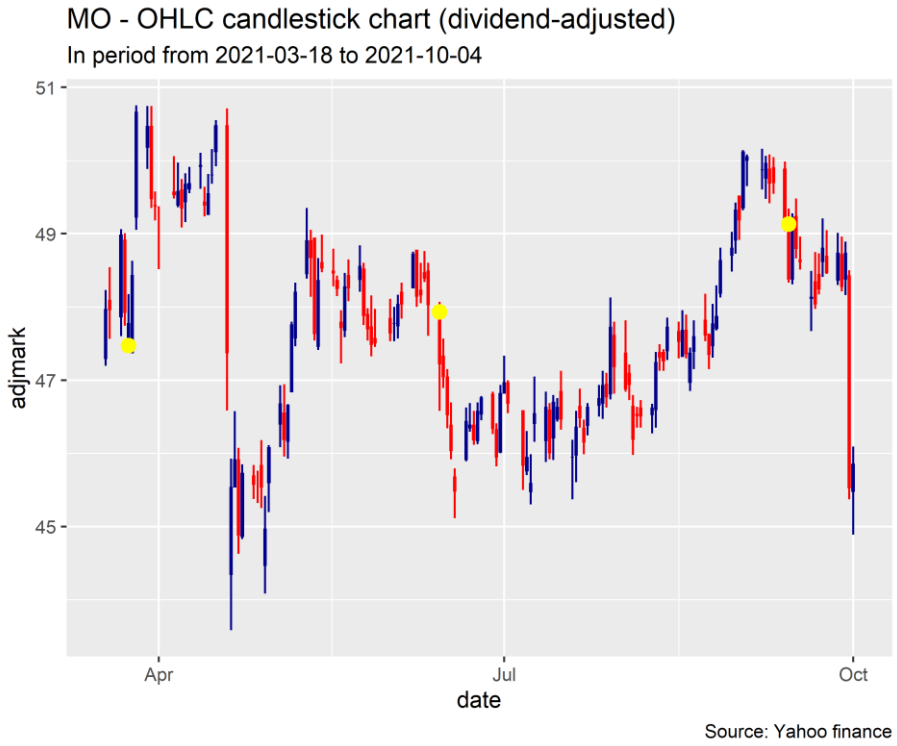
```
# Dividend-adjusted OHLC prices
prices2 <- prices2 %>%
  mutate(adjfactor = adjusted/close,
         adjopen = adjfactor * open,
         adjhigh = adjfactor * high,
         adjlow = adjfactor * low,
         adjclose = adjusted,
         adjmark = adjopen*(adj_dividend>0))
```

- Where the newly created variable **adjmark** variable was shifted to the **adjopen** price in order to be consistend with the adjusted OHLC chart

MO – Candlestick chart

- We can also plot the adjusted OHLC prices of AAPL on a **candlestick chart**:

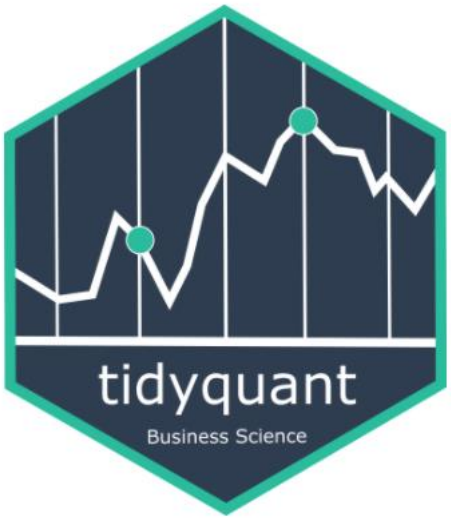
```
# Plot OHLC candlestick stock prices (last 200 days)
prices2 %>%
  filter(date >= tEnd-kDays) %>%
  ggplot(aes(x = date)) +
  geom_candlestick(aes(open = adjopen, high = adjhigh, low = adjlow, close = adjclose)) +
  geom_point(aes(y = adjmark),size = 3, color="yellow") +
  labs(title = paste0(symbol, " - OHLC candlestick chart (dividend-adjusted)"),
       subtitle = paste0("In period from ",tEnd-kDays," to ",tEnd),
       caption = "Source: Yahoo finance")
```



- We can see that the **dividend-adjusted OHLC prices** of Altria do not exhibit such significant jumps of the opening price on the ex-dividend date anymore
- These prices are thus suitable for assesing of the stock performance, calculation of technical indicators, and other similar applications

Introduction to Tidyquant

- ✓ Introduction to R packages
- ✓ Introduction to Tidyverse
- ➢ **Introduction to Tidyquant**
  - ✓ Basic Tidyquant functions
  - ✓ Processing stock price data
  - ➢ **Case study 1 – S&P500 crashes**
  - ❖ Processing data for multiple stocks
  - ❖ Case study 2 – Factor investing



Case Study 1 – S&P 500 crashes

- The goals of the case study are:
  1. Create a dataset of the largest 1-day crashes of S&P500
  2. Analyse the expected returns in day t+1 following a S&P500 crash in day t
  3. Verify statistical signifikance of the expected returns
  4. Assess the signifikance of the „Monday Reversal Effect“

- Steps:
  A. Dataset preparation and analysis
  B. Analysis of returns after largest 1-day crashes
  C. Analysis of the „Monday Reversal Effect“

Case Study 1 – S&P 500 crashes
Part A: Dataset preparation

Load S&P 500 prices

- We start by downloading prices of the S&P 500 stock price index

```
# Settings:
symbol = "^GSPC"
tStart <- "1900-01-01"
tEnd   <- today()
# Load stock prices:
data <- tq_get(symbol, get = "stock.prices", from = tStart, to = tEnd)
```

- Benefits and drawbacks of the S&P 500 time-series (^GSPC):
  - (+) Very long time-series (starts since 30.12.1927)
  - (-) Yahoo Finance data on stock indices do not contain dividend adjustments
  - (-) Is not directly tradeable (can be approximated with CFD, ETF, futures, etc.)
- Alternative → Use data of an of an ETF that tracks S&P 500 (such as SPY):
  - (-) Shorter time-series (starts around 1993)
  - (+) Yahoo Finance reports dividend-adjusted prices (column „adjusted“)
  - (+) Directly tradeable
- To download SPY just replace `"^GSPC"` in the code with `"SPY"`

Load S&P 500 prices

- We start by downloading prices of the S&P 500 stock price index

```
# Settings:
symbol = "^GSPC"
tStart <- "1900-01-01"
tEnd   <- today()
# Load stock prices:
data <- tq_get(symbol, get = "stock.prices", from = tStart, to = tEnd)
```

- Benefits and drawbacks of the S&P 500 time-series (^GSPC):
  - (+) Very long time-series (starts since 30.12.1927)
  - (-) Yahoo Finance data on stock indices do not contain dividend adjustments
  - (-) Is not directly tradeable (can be approximated with CFD, ETF, futures, etc.)
- Alternative → Use data of an of an ETF that tracks S&P 500 (such as SPY):
  - (-) Shorter time-series (starts around 1993)
  - (+) Yahoo Finance reports dividend-adjusted prices (column „adjusted“)
  - (+) Directly tradeable
- To download SPY just replace `"^GSPC"` in the code with `"SPY"`
- In the case study we will work with the index (^GSPC)

S&P 500 – Visual analysis

- We will start by compute S&P 500 daily returns some derived time-series:

```
# Compute returns and other time-series
data <- data %>%
  mutate(logprice = log(adjusted),
         logreturn = logprice - lag(logprice,1),
         logreturn = ifelse(is.na(logreturn),0,logreturn),
         return = exp(logreturn)-1,
         cumreturn = cumprod(1+return),
         drawdown = (cumreturn/cummax(cumreturn))-1)
```

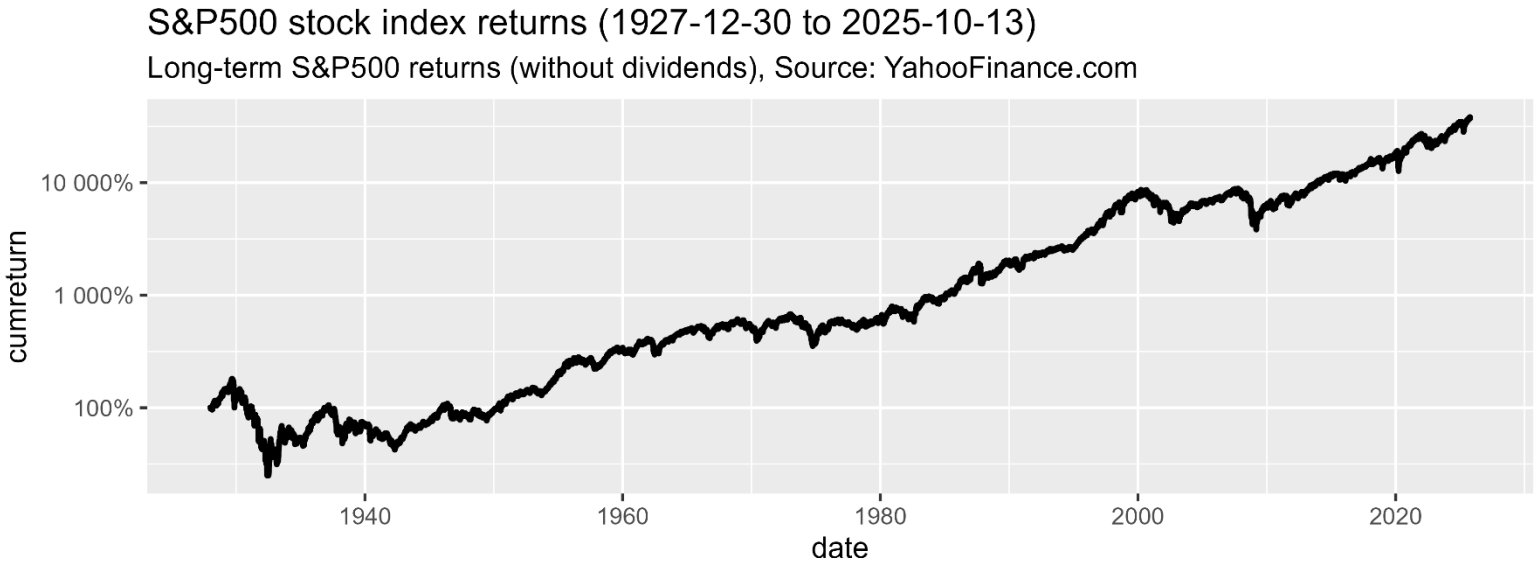
- Where:
  - **logprice** = Logarithm of the S&P 500 index close price in day t
  - **logreturn** = Logarithmic return (i.e. difference of the logprice)
  - **return** = Simple return, i.e. $return_t = \frac{Close_t}{Close_{t-1}} - 1 = exp^{logreturn_t} - 1$
  - **cumreturn** = Cumulative return of an investment into S&P 500
  - **drawdown** = Largest negative return since previously achieved all-time-high

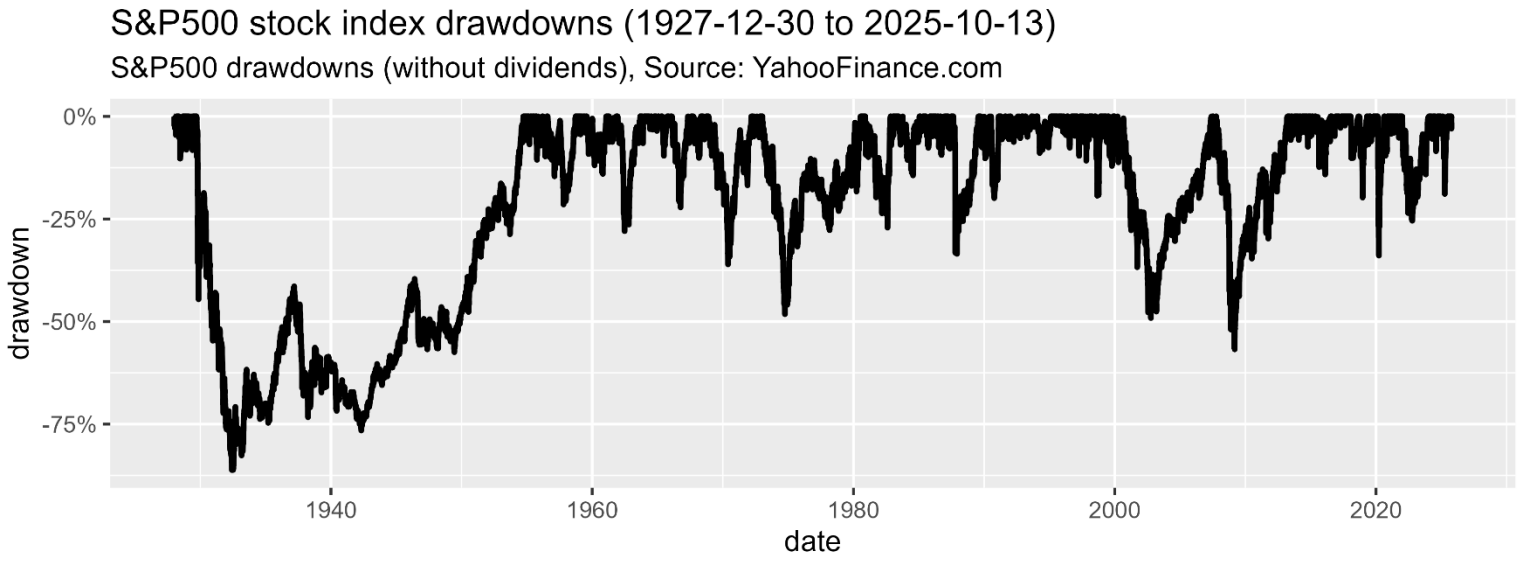
Generate S&P 500 return charts

- Plot S&P 500 cumulative returns and drawdowns

```
# Identify dataset start and end
dStart <- min(data$date)
dEnd <- max(data$date)
# Plot cumulative returns
gChart01 <- data %>%
  ungroup() %>%
  ggplot(aes(x = date, y = cumreturn)) +
  geom_line(size = 1) +
  labs(title = paste0("S&P500 returns (",dStart," to ",dEnd,")"),
       subtitle = paste0("Source: YahooFinance.com")) +
  scale_y_continuous(labels = scales::percent,trans='log10')
print(gChart01)
# Plot drawdowns
gChart02 <- data %>%
  ungroup() %>%
  ggplot(aes(x = date, y = drawdown)) +
  geom_line(size = 1) +
  labs(title = paste0("S&P500 drawdowns (",dStart," to ",dEnd,")"),
       subtitle = paste0("Source: YahooFinance.com")) +
  scale_y_continuous(labels = scales::percent)
print(gChart02)
```

S&P 500 returns & drawdowns





Case Study 1 – S&P 500 crashes
Part B: Analysis of 1-day crashes

Compute additional series

- First let us compute some context variables (lagged returns, day of week, etc.):

```
# Compute lagged and lead returns
data <- data %>%
  mutate(target = lead(return, default = 0),        # return in day t+1
         lagreturn = lag(return, default = 0),      # return in day t-1
         lagreturn2 = lag(return, 2, default = 0), # return in day t-2
         lagdrawdown = lag(drawdown, default = 0), # drawdown in day t-1
         wday = weekdays(date))                     # day of the week
```

- For easier exhibition we will also sort the dataset by **return** in day **t**:

```
# Sort dataset by return in day t
dataS <- data %>%
  select(date,wday,lagdrawdown,drawdown,
         lagreturn2,lagreturn,return,target) %>%
  arrange(return)
```

Filter out largest 1-day crashes

- Specify a return threshold (i.e. -5% daily return):

```
# Specify threshold
threshold <- -0.05 # threshold on return(t)
```

- Filter out observations where **return(t) < threshold**

```
# Filter out relevant crashes
temp <- dataS %>%
  filter(return<=threshold)
```

- Compute number of crashes (for illustration):

```
# Compute dataset size
nCrash <- nrow(temp)
nCrash
```

- Print the tibble containing information on all crashes

```
# Show data
print(temp,n=nCrash)
```

Largest 1-day S&P 500 crashes

- Specify a return threshold (i.e. -5% daily return):

```
# Specify threshold
threshold <- -0.05 # threshold on return(t)
```

- Filter out observations where **return(t) < threshold**

```
# Filter out relevant crashes
temp <- dataS %>%
  filter(return<=threshold)
```

- Compute number of crashes (Note: in total there is **87** crashes of more than **-5%**):

```
# Compute dataset size
nCrash <- nrow(temp)
nCrash
```

- Print the tibble containing information on all crashes

```
# Show data
print(temp,n=nCrash)
```

20 largest 1-day S&P 500 crashes

A tibble: 87 × 8

	date	wday	lagdrawdown	drawdown	lagreturn2	lagreturn	return	target
	<date>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1987-10-19	Monday	-0.161	-0.332	-0.0234	-0.0516	-0.205	0.0533
2	1929-10-28	Monday	-0.180	-0.286	-0.0320	0.0144	-0.129	-0.102
3	2020-03-16	Monday	-0.199	-0.295	-0.0951	0.0929	-0.120	0.0600
4	1929-10-29	Tuesday	-0.286	-0.359	0.0144	-0.129	-0.102	0.125
5	1935-04-16	Tuesday	-0.713	-0.742	0.0113	0.0201	-0.0997	0.0961
6	1929-11-06	Wednesday	-0.282	-0.353	0.0505	-0.0526	-0.0992	0.0369
7	1946-09-03	Tuesday	-0.477	-0.529	0.00179	-0.00656	-0.0991	0.0307
8	2020-03-12	Thursday	-0.190	-0.267	0.0494	-0.0489	-0.0951	0.0929
9	1937-10-18	Monday	-0.628	-0.662	-0.0197	-0.0109	-0.0912	0.0316
10	1931-10-05	Monday	-0.696	-0.723	-0.0185	0.0178	-0.0907	0.124
11	2008-10-15	Wednesday	-0.362	-0.420	0.116	-0.00532	-0.0903	0.0425
12	2008-12-01	Monday	-0.427	-0.479	0.0353	0.00964	-0.0893	0.0399
13	1933-07-20	Thursday	-0.636	-0.668	0.0158	-0.0492	-0.0888	-0.0870
14	2008-09-29	Monday	-0.225	-0.293	0.0197	0.00338	-0.0881	0.0542
15	1933-07-21	Friday	-0.668	-0.697	-0.0492	-0.0888	-0.0870	0.0881
16	1932-10-10	Monday	-0.780	-0.798	0.00541	-0.0552	-0.0855	0.0717
17	1987-10-26	Monday	-0.263	-0.324	-0.0392	-0.000121	-0.0828	0.0242
18	1932-10-05	Wednesday	-0.747	-0.768	-0.00248	-0.00124	-0.0820	0.00541
19	1932-08-12	Friday	-0.761	-0.780	0.0352	-0.00523	-0.0802	0.0629
20	1935-08-16	Friday	-0.638	-0.667	0	-0.0111	-0.0797	0.0753

- Black Monday (1987/10/19) → SPY droped by 20.5% in a single day
- Other prominent events → Great Depression (1929-1933), GFC (2008), Covid (2020)
- Next-day return (t+1) → Almost always positive (!)

Expected return + significance

- Prepare function to compute summary statistics

```
# Prepare function to compute summary statistics
# Including a t-test with H0: E(target)=0 and H1: E(target)!=0
csummary <- function(temp) {
  crash_summary <- temp %>%
    summarize(nObs = n(),
              mju = mean(target),
              sigma = sd(target),
              tStat = mju/(sigma/(sqrt(nObs))),
              pValue = 2*(1-pt(abs(tStat),df=nObs-1)))
  return(crash_summary)
}
```

- Compute summary statistics and t-test for **H0: E(target) = 0**

```
# Compute summary
crash_summary <- csummary(temp)
crash_summary
```

- Note – You can alternativelly compute the t-test with:

```
# Check of the t-test with R
t.test(temp$target)
```

Expected return + significance

- Prepare function to compute summary statistics

```
# Prepare function to compute summary statistics
# Including a t-test with H0: E(target)=0 and H1: E(target)!=0
csummary <- function(temp) {
  crash_summary <- temp %>%
    summarize(nObs = n(),                                  # Observations
              mju = mean(target),                          # Average return
              sigma = sd(target),                          # St. deviation
              tStat = mju/(sigma/(sqrt(nObs))),            # t-statistics
              pValue = 2*(1-pt(abs(tStat),df=nObs-1)))     # p-value
  return(crash_summary)
}
```

- Compute summary statistics and t-test for **H0: E(target) = 0**

```
# Compute summary
crash_summary <- csummary(temp)
crash_summary
```

- Result → Average return **1.32%** with **t-stat=2.33** and **p-value=2.22%**

```
> crash_summary
# A tibble: 1 × 5
   nObs     mju  sigma tStat pValue
  <int>   <dbl>  <dbl> <dbl>  <dbl>
1    87 0.0132 0.0530  2.33 0.0222
```

Interpretation: The chance that we would observe this result if H0 was trueis only 2.22% → **We reject H0**

Robustness check – Post-1990 data

- Re-calculate the result on **post-1990 data** where the market were more efficient

```
# Start date
dThreshold <- "1990-01-01"
# Filter dataset
temp2 <- temp %>%
  filter(date>=dThreshold)
# Compute dataset size
nCrash <- nrow(temp2)
nCrash
# Show data
print(temp2,n=nCrash)
# Compute summary
crash_summary <- csummary(temp2)
crash_summary
```

- Result → Average return **2.73%** with **t-stat=3.33** and **p-value=0.319%**

```
> crash_summary
# A tibble: 1 × 5
   nObs    mju  sigma tStat  pValue
  <int>  <dbl>  <dbl> <dbl>   <dbl>
1    22 0.0273 0.0385  3.33 0.00319
```

- Higher significance (p-value) with only **22 observations**
- **Note:** Take note the t-test assumes Gaussian returns and *i.i.d.* → Not realistic

Illustration – Post-1990 crashes

A tibble: 22 × 8

	date	wday	lagdrawdown	drawdown	lagreturn2	lagreturn	return	target
	<date>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2020-03-16	Monday	-0.199	-0.295	-0.0951	0.0929	-0.120	0.0600
2	2020-03-12	Thursday	-0.190	-0.267	0.0494	-0.0489	-0.0951	0.0929
3	2008-10-15	Wednesday	-0.362	-0.420	0.116	-0.00532	-0.0903	0.0425
4	2008-12-01	Monday	-0.427	-0.479	0.0353	0.00964	-0.0893	0.0399
5	2008-09-29	Monday	-0.225	-0.293	0.0197	0.00338	-0.0881	0.0542
6	2008-10-09	Thursday	-0.371	-0.419	-0.0574	-0.0113	-0.0762	-0.0118
7	2020-03-09	Monday	-0.122	-0.189	-0.0339	-0.0171	-0.0760	0.0494
8	1997-10-27	Monday	-0.0422	-0.108	-0.0184	-0.00952	-0.0687	0.0512
9	1998-08-31	Monday	-0.134	-0.193	-0.0384	-0.0148	-0.0680	0.0386
10	2008-11-20	Thursday	-0.485	-0.519	0.00984	-0.0612	-0.0671	0.0632
11	2011-08-08	Monday	-0.234	-0.285	-0.0478	-0.000575	-0.0666	0.0474
12	2008-11-19	Wednesday	-0.451	-0.485	-0.0258	0.00984	-0.0612	-0.0671
13	2008-10-22	Wednesday	-0.390	-0.427	0.0477	-0.0308	-0.0610	0.0126
14	2025-04-04	Friday	-0.122	-0.174	0.00673	-0.0484	-0.0597	-0.00233
15	2020-06-11	Thursday	-0.0579	-0.113	-0.00780	-0.00531	-0.0589	0.0131
16	2000-04-14	Friday	-0.0569	-0.112	-0.0223	-0.0182	-0.0583	0.0331
17	2008-10-07	Tuesday	-0.325	-0.363	-0.0135	-0.0385	-0.0574	-0.0113
18	2009-01-20	Tuesday	-0.457	-0.486	0.00133	0.00756	-0.0528	0.0435
19	2008-11-05	Wednesday	-0.357	-0.391	-0.00253	0.0408	-0.0527	-0.0503
20	2008-11-12	Wednesday	-0.426	-0.455	-0.0127	-0.0220	-0.0519	0.0692
21	2020-03-18	Wednesday	-0.253	-0.292	-0.120	0.0600	-0.0518	0.00471
22	2008-11-06	Thursday	-0.391	-0.422	0.0408	-0.0527	-0.0503	0.0289

- Main events: dot.com bubble (2000), GFC (2008), Covid (2020), tariffs (2025)
- Returns in t+1 are overwhelmingly positive

Post-crash returns – Explanations

- Why does the anomaly occurr – Different interpretations:
  1. **Compensation for tail-risk** → Buying after a market drop can be risky → Market risk premium increases → Example: In 1987-10-16 the market dropped -5.16% and the next day -20.5%
  2. **Returns from liquidity provision** → The ones who sell during crashes bear the „costs of immediacy“ (i.e. they want to sell quickly when many investors are selling) → The ones who are willing to buy recieve „returns from liquidity provision“ (i.e. for allowing others to sell)
  3. **Market over-reaction** → Some investors may over-react to negative news (i.e. Behavioural biases)
  4. **Statistical explanations** → Few observations (especially of extreme tail-risk events such as the 1987-10-17 market crash) + the t-test assumes Gaussian returns and *i.i.d.* (i.e. constant volatility)

Case Study 1 – S&P 500 crashes
Part C: Monday Reversal Effect

Monday Reversal Effect

- Prominent market anomalies occurr around Monday (i.e. after the weekend):
  - **Monday Effect** → Stock returns tend to be worse on Monday then any other day of the week → The effect has mostly disappeared on 1990 data
  - **Monday Reversal Effect** → Stock returns on Monday (especially negative returns) tend to reverse during Tuesday (Bessembindel and Hertzel, 1993, Ülkü and Andonov, 2016) → The anomaly seems to still be present
- Lets verify the presence of both effects in our dataset

Monday effect – 1927-2025

- Compute **summary** statistics for **each day of the week**:

```
# Illustration - Monday effect
weekday_returns <- data %>%
  group_by(wday) %>%
  summarize(nObs = n(),
            mju = mean(return),
            sigma = sd(return),
            tStat = mju/(sigma/(sqrt(nObs))),
            pValue = 2*(1-pt(abs(tStat),df=nObs-1)))
weekday_returns
```

- **Result** → Monday is the only weekday with **negative average returns**

```
> weekday_returns
# A tibble: 5 × 6
  wday       nObs       mju  sigma tStat     pValue
  <chr>     <int>     <dbl>  <dbl> <dbl>      <dbl>
1 Friday     4911  0.000532 0.0111  3.35 0.000801
2 Monday     4724 -0.000652 0.0138 -3.25 0.00116
3 Thursday   4932  0.000402 0.0112  2.53 0.0115
4 Tuesday    4989  0.000459 0.0116  2.80 0.00508
5 Wednesday  5006  0.000775 0.0119  4.61 0.00000421
```

Monday effect – 1990-2025

- Compute **summary** statistics for **each day of the week** on **post-1990 data**:

```
# Illustration - Monday effect - Since 1990
dThreshold <- "1990-01-01"
weekday_returns <- data %>%
  filter(date>=dThreshold) %>%
  group_by(wday) %>%
  summarize(nObs = n(),
            mju = mean(return),
            sigma = sd(return),
            tStat = mju/(sigma/(sqrt(nObs))),
            pValue = 2*(1-pt(abs(tStat),df=nObs-1)))
weekday_returns
```

- **Result** → Monday is no worse than other weekday → Monday effect disappeared

```
> weekday_returns
# A tibble: 5 × 6
  wday       nObs      mju  sigma tStat pValue
  <chr>     <int>    <dbl>  <dbl> <dbl>  <dbl>
1 Friday     1804 0.000210 0.0107 0.835 0.404
2 Monday     1698 0.000429 0.0124 1.42  0.155
3 Thursday   1815 0.000185 0.0115 0.686 0.493
4 Tuesday    1849 0.000628 0.0114 2.36  0.0182
5 Wednesday  1846 0.000497 0.0110 1.95  0.0517
```

Monday Reversal – 1927-1925

- Compute **summary** statistics for **each day of the week**:

```
# Specify thresholds
threshold <- -0.02    # threshold on return(t)
xwday <- "Monday"     # select day
# Show largest crashes
temp <- dataS %>%
  filter(return<=threshold) %>%
  filter(wday==xwday)
# Compute summary
crash_summary <- csummary(temp)
crash_summary
```

- **Result → 257** observations, **0.56% average return** with **t-stat=3.72**

```
> crash_summary
# A tibble: 1 × 5
   nObs      mju  sigma tStat   pValue
  <int>    <dbl>  <dbl> <dbl>    <dbl>
1   257 0.00558 0.0240  3.72 0.000244
```

Monday Reversal – 1990-2025

- Compute **summary** statistics for **each day of the week (post-1990)**:

```
# Start date
dThreshold <- "1990-01-01"
# Filter dataset
temp2 <- temp %>%
  filter(date>=dThreshold)
# Compute summary
crash_summary <- csummary(temp2)
crash_summary
```

- **Result → 68** observations, **1.11% average return** with **t-stat=3.51**

```
> crash_summary
# A tibble: 1 × 5
   nObs    mju  sigma tStat   pValue
  <int>  <dbl>  <dbl> <dbl>    <dbl>
1    68 0.0112 0.0264  3.51 0.000819
```

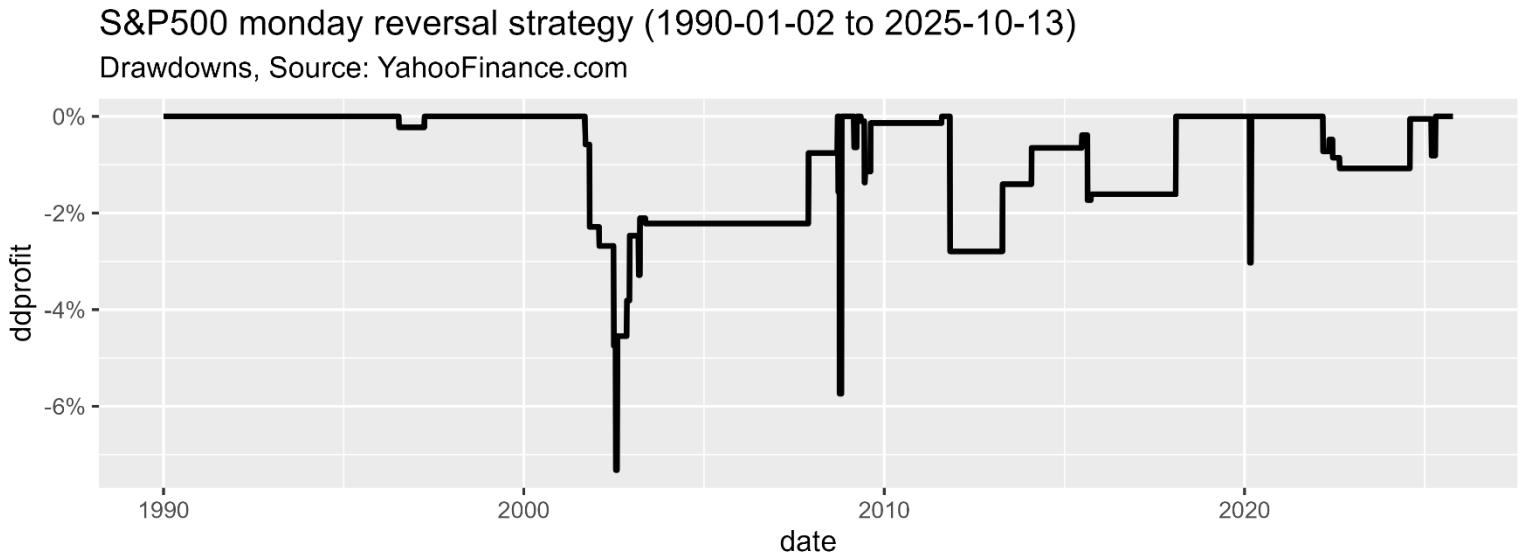
Compute cumulative returns

- Compute **cumulative ret.** and **drawdowns** of trading based on Monday reversals

```
# Compute strategy returns
data2 <- data %>%
  filter(date>=dThreshold) %>%
  mutate(entry = (return<=threshold) & (wday==xwday),
         profit = entry*target,
         cumprofit = cumprod(1+profit),
         ddprofit = (cumprofit/cummax(cumprofit))-1)
# Plot cumulative returns
gChart03 <- data2 %>%
  ungroup() %>%
  ggplot(aes(x = date, y = cumprofit)) +
  geom_line(size = 1) +
  labs(title = paste0(„Monday reversal strategy (",dStart," to ",dEnd,")"),
       subtitle = paste0("Cumulative returns, Source: YahooFinance.com")) +
  scale_y_continuous(labels = scales::percent)
print(gChart03)
# Plot drawdowns
gChart04 <- data2 %>%
  ungroup() %>%
  ggplot(aes(x = date, y = ddprofit)) +
  geom_line(size = 1) +
  labs(title = paste0(„Monday reversal strategy (",dStart," to ",dEnd,")"),
       subtitle = paste0("Drawdowns, Source: YahooFinance.com")) +
  scale_y_continuous(labels = scales::percent)
print(gChart04)
```

Cumulative returns & drawdowns



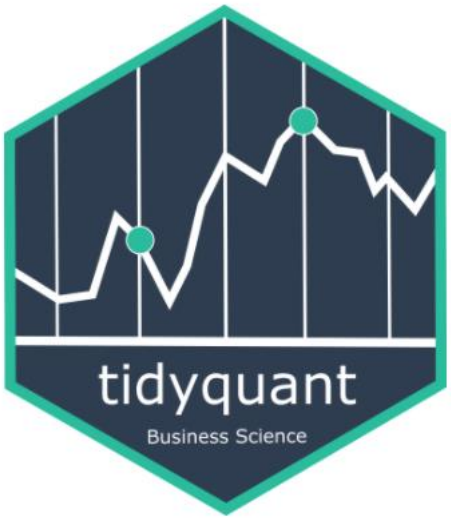


Monday Reversal – Conclusions

- Assessment of the Monday Reversal strategy:
  - (+) Persistent anomaly (robust over ~100 years of history)
  - (+) Highest returns during crisis periods (2008, 2020, etc.)
  - (-) Provides trading signals only rarely (68 times since 1990)
  - (-) Potentially high tail-risk (buying after a crash)

Introduction to Tidyquant

- ✓ Introduction to R packages
- ✓ Introduction to Tidyverse
- ➢ **Introduction to Tidyquant**
  - ✓ Basic Tidyquant functions
  - ✓ Processing stock price data
  - ✓ Case study 1 – S&P500 crashes
  - ➢ **Processing data for multiple stocks**
  - ❖ Case study 2 – Factor investing



Processing data for multiple stocks

- **Task:** Download the OHLCV data for the 4 most traded USD stocks (AAPL, MSFT, GOOG, AMZN) in the time period since 1.1.2011 until today

- We set the **settings** as follows:

```
# Settings:
symbols <- c("AAPL","MSFT","GOOG","AMZN")
tStart <- "2011-01-01"
tEnd   <- today()
```

- And load the stock data by using:

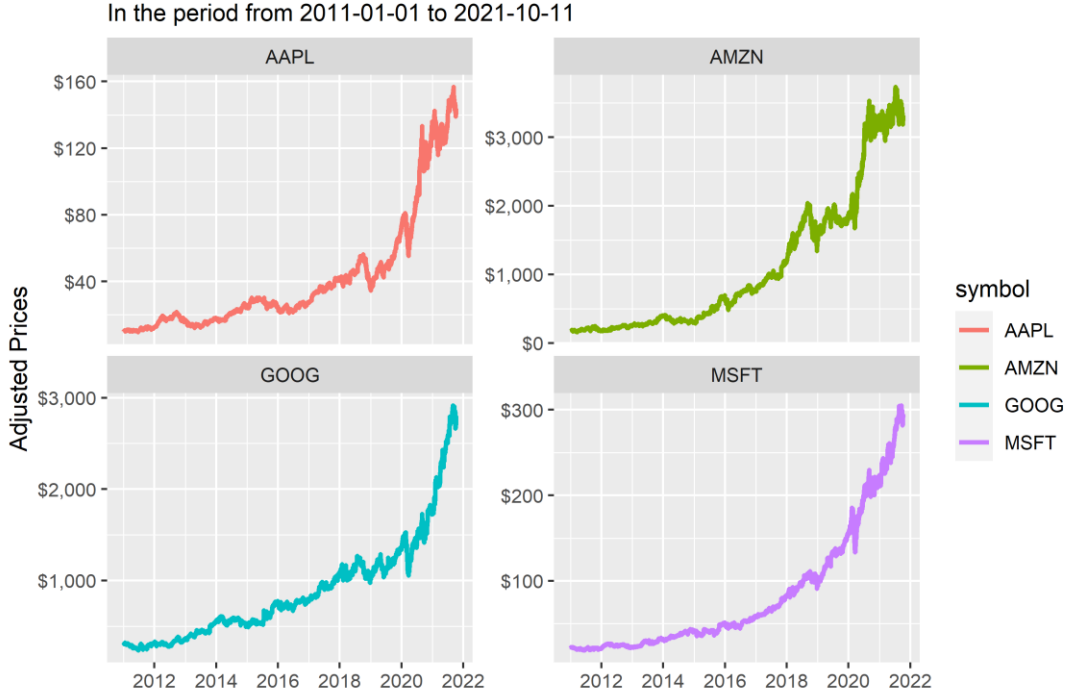
```
# Load stock prices
prices <- tq_get(symbols, get = "stock.prices", from = tStart, to = tEnd)
```

- The stock prices will be downloaded into a **tibble** in the **long format**

Plotting data for multiple stocks

- We can plot stock prices for multiple stocks by using the **group_by** statement:

```
# Plot adjusted stock price evolution for multiple stocks
prices %>%
  group_by(symbol) %>%
  ggplot(aes(x = date, y = adjusted, color = symbol)) +
  geom_line(size = 1) +
  labs(title = "Daily Stock Prices",
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       x = "", y = "Adjusted Prices") +
  facet_wrap(~ symbol, ncol = 2, scales = "free_y") +
  scale_y_continuous(labels = scales::dollar)
```

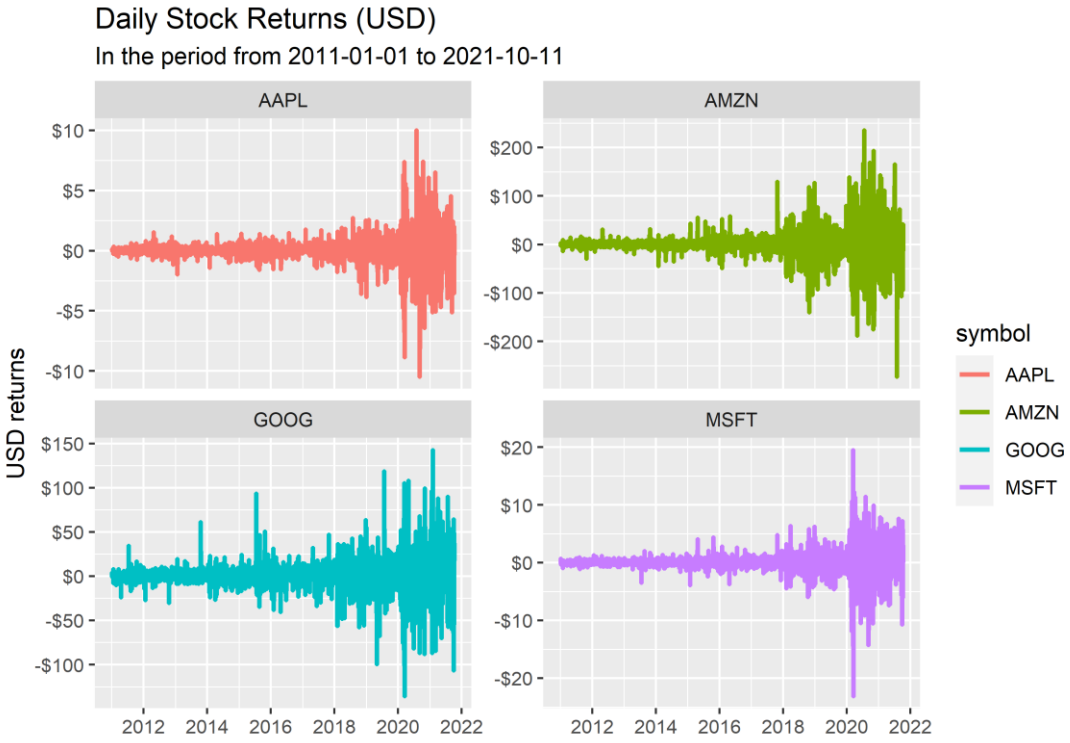


- The **group_by** statement groups the stocks based on **symbol**, so that they are subsequently recognized by ggplot as separate time series
- With the **facet_wrap** statement we then told R to plot the stocks on 4 separate charts (instead of a single chart)
- With the **scale_y_continuous** statement we then set that the y scale should be shown in $

Calculation of USD returns

- We can calculate **daily USD returns** of the stocks by using functions:
  - **group_by** = To tell R that the calculation should be performed separatelly for each group (symbol)
  - **mutate** = To create a new column (USD return) via transformation of other columns
  - **lag** = To lag the adjusted close price in order to be able to compute the USD returns (differences)
- The calculation can be done as follows (we do not show the plotting):

```
# Compute USD returns
prices <- prices %>%
  group_by(symbol) %>%
  mutate(return_USD = adjusted - lag(adjusted))
```



- We can see that the time series of USD returns exhibit significant **heteroskedasticity** (time-varying variance)
- This is caused by the fact that the magnitude of price movements depends on the price level
- USD returns are thus **not suitable for econometric analysis**
- Much more suitable for econometric modelling are the **relative (%) returns**

Calculation of relative returns

- We can compute the **relative (%) returns** as follows:

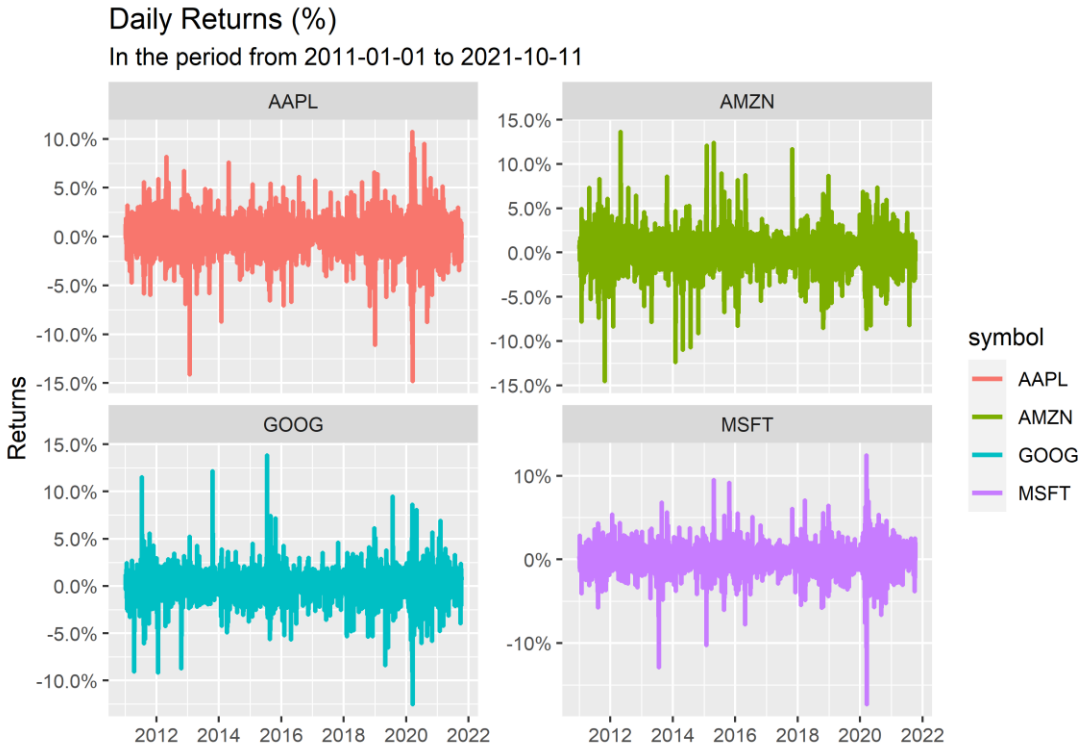
```
# Compute and daily % returns
prices <- prices %>%
  group_by(symbol) %>%
  mutate(return = (adjusted - lag(adjusted)) / lag(adjusted))
```

- In order to plot them on a chart with % on the y axis, we need to replace:

```
scale_y_continuous(labels = scales::dollar)
```

- With:

```
scale_y_continuous(labels = scales::percent)
```



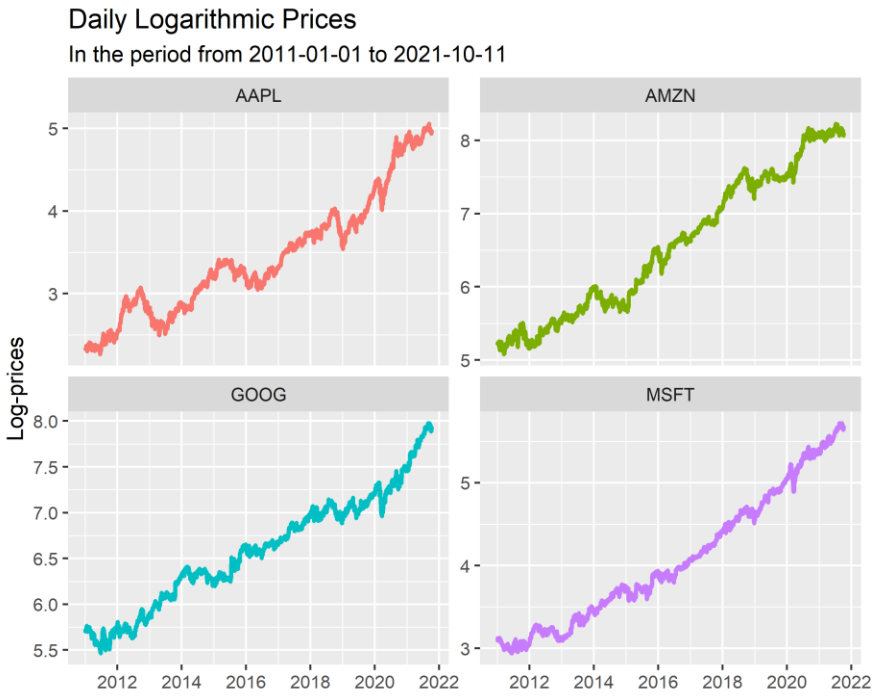
- The relative returns still exhibit some **heteroskedasticity** due to **volatility clustering**, but it is much lesser than for the USD returns
- Relative returns can also be approximated with **logarithmic returns**, which are easier to work with as they can be added in order to compute the linearized cumulative return of the stock

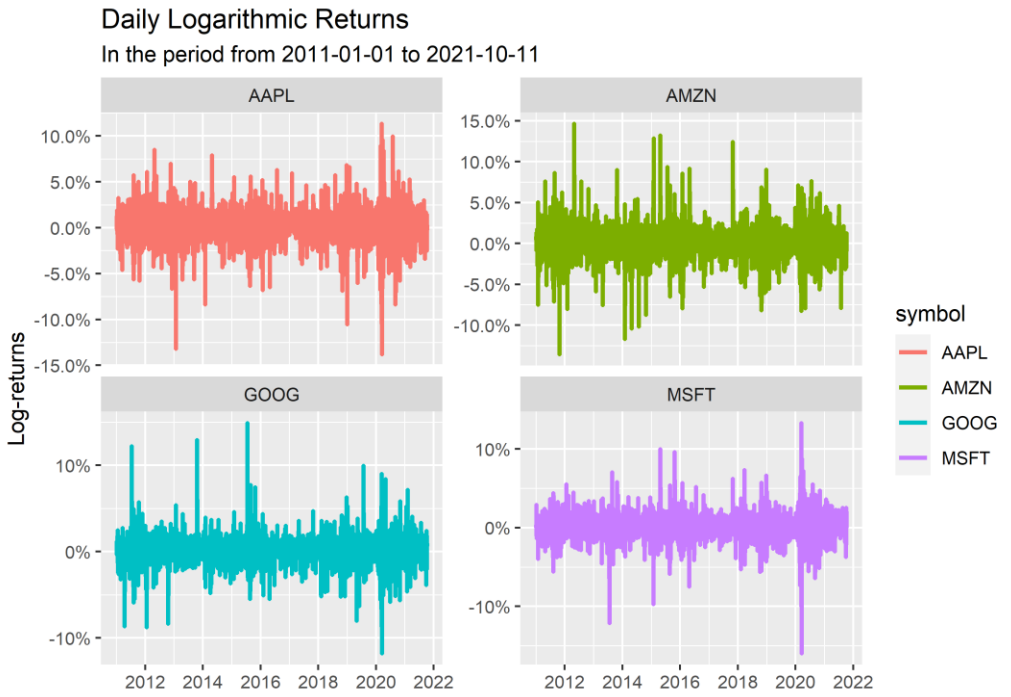
Logarithmic returns

- We can compute the **logarithmic stock prices** and **logarithmic returns** as follows:

```
# Compute logarithmic stock prices and daily logarithmic returns
prices <- prices %>%
  group_by(symbol) %>%
  mutate(logprice = log(adjusted),
         logreturn = logprice - lag(logprice))
```

- **Logarithmic prices** show us the **linearized** evolution of the stock price (i.e. they transform the exponential evolution of the stock price into a linear one)
- **Logarithmic returns** are very close to % returns and are often used as their **approximation**



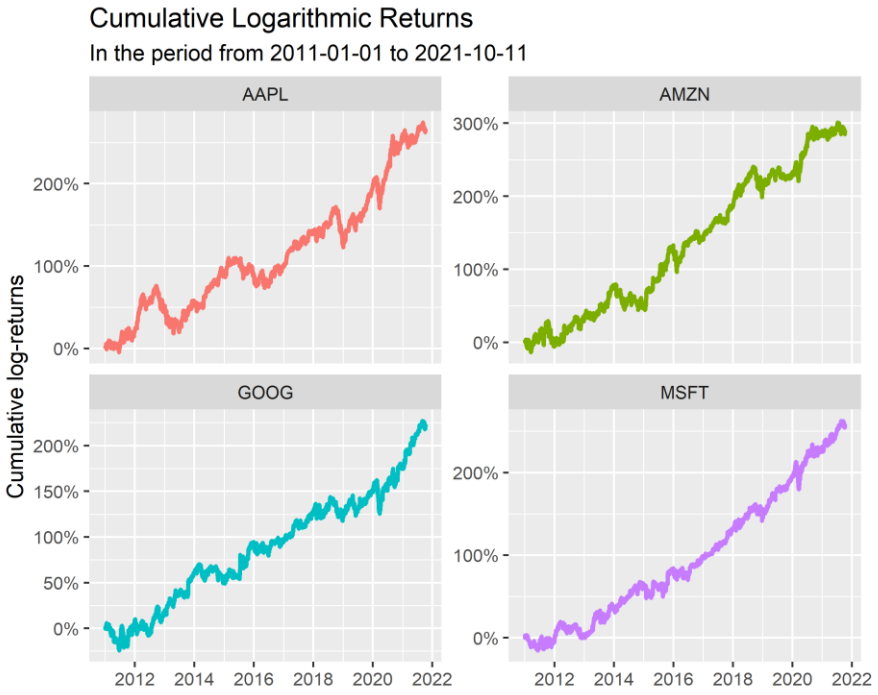


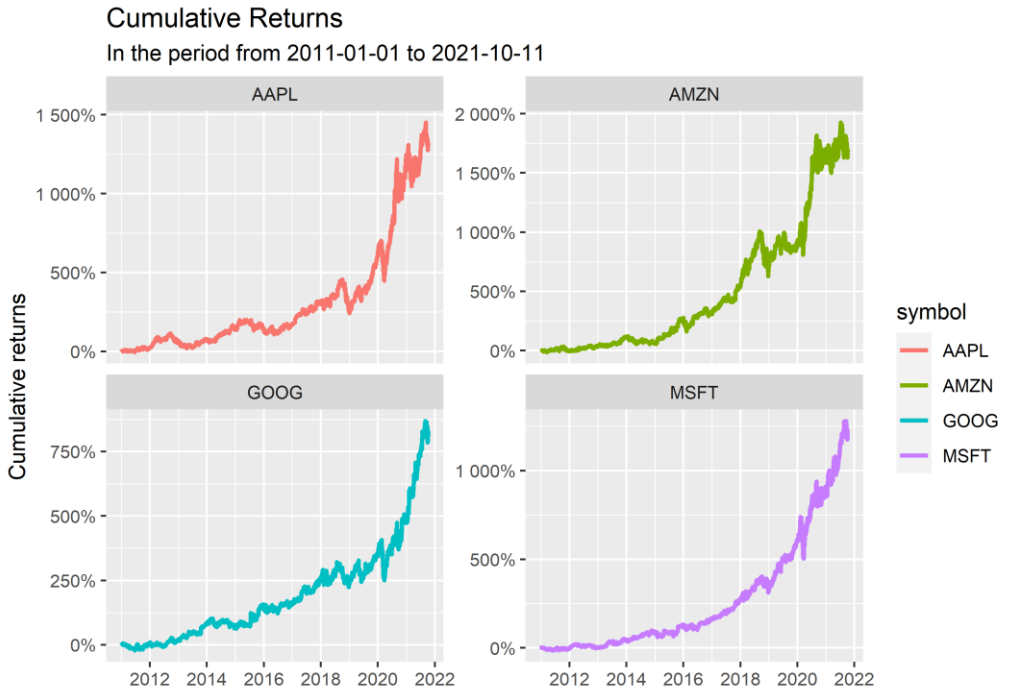
Cumulative returns

- We can compute **cumulative logarithmic returns** and **cumulative total returns** as follows:

```
# Cumulative logarithmic returns
prices <- prices %>%
  group_by(symbol) %>%
  mutate(logreturn = ifelse(is.na(logreturn),0,logreturn),
         cumlogret = cumsum(logreturn),
         cumreturn = exp(cumlogret)-1)
```

- **Cumulative logarithmic returns** tell us the cumulative **linearized** return of the stock (i.e. return that we would achieve without „reinvestment“ of the previous gains)
- Logarithmic returns can also be easily transformed to **exponentially growing cumulative returns**



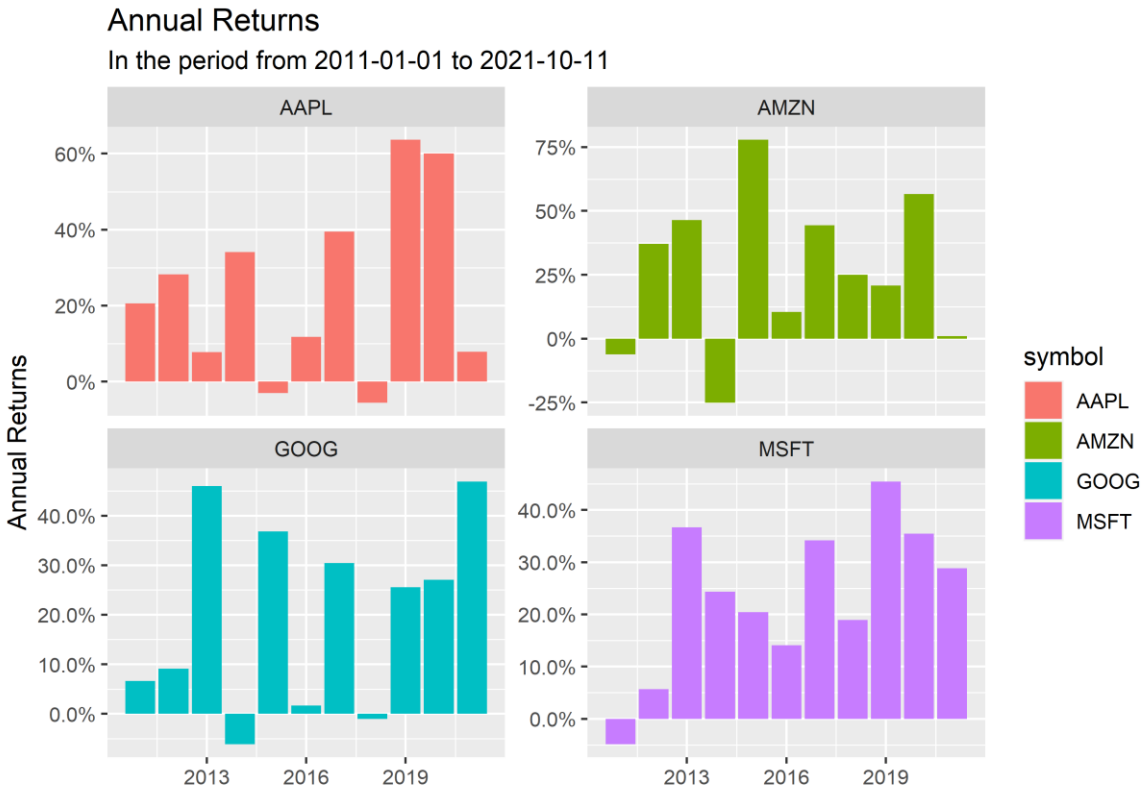


Annual returns

- We can also compute **annual logarithmic returns** by grouping the data by symbol and year, and computing the sum of daily returns for each year with the function **summarize**:

```
# Calculate annual logarithmic returns
yprices <- prices %>%
  mutate(year = year(date)) %>%
  group_by(symbol,year) %>%
  summarize(return = sum(logreturn))
```

- To plot the returns on a chart, we need to use **geom_col()** in the ggplot settings



- We can see that all of the stocks were providing high positive returns (around 20%) in most of the years since 2011
- While the logarithmic returns slightly differ from the actual percentage returns, the differences are very small, and the logarithmic returns are easier to work with as we can sum them over time, as well as between different stocks (to form a portfolio)

Download stock index components

- **Task:** Download DJIA stock index components
- The simplest way to do that is by using the **tq_index** function that tells us the components of a selected stock index
- The currently supported stock indices can be found with **tq_index_options:**

```
# Supported stock indices
tq_index_options()
[1] "DOW"       "DOWGLOBAL" "SP400"     "SP500"     "SP600"
```

- We can thus get the current components of the **DJIA stock index** by calling:

```
# Get DJIA components
info <- tq_index("DOW")
```

- The tibble **info** will look as follows (for the first 5 stocks):

	symbol	company	identifier	sedol	weight	sector	shares_held	local_currency
1	UNH	UnitedHealth Group Incorporated	91324P10	2917766	0.076665741	Health Care	5563210	USD
2	GS	Goldman Sachs Group Inc.	38141G10	2407966	0.073998439	Financials	5563210	USD
3	HD	Home Depot Inc.	43707610	2434209	0.063931686	Consumer Discretionary	5563210	USD
4	MSFT	Microsoft Corporation	59491810	2588173	0.055855915	Information Technology	5563210	USD
5	CRM	salesforce.com inc.	79466L30	2310525	0.052016001	Information Technology	5563210	USD

- Where: **symbol** = stock symbol, **company** = firm name, **weight** = weight in the stock index, **sector** = sector of the firm, **shares_held** = number of shares in the stock index, **local_currency** = denomination of the stock

Download all stocks in an index

- **Task:** Download the OHLCV data for all of the stocks in the DJIA stock index in the period from 1.1.2011 until today
- We can do that easily by combining **tq_get** and **tq_index**
- First, we downalod the **components** of the DJIA stock index with **tq_index:**

```
# Get DJIA components
info <- tq_index("DOW")
```

- We then set the remaining **settings** of the task:

```
# Settings:
tStart <- "2011-01-01"
tEnd   <- today()
```

- Finally, we download all of the OHLCV prices with **tq_get:**

```
# Load DJIA stock prices
prices <- tq_get(info$symbol, get = "stock.prices", from = tStart, to = tEnd)
```

- We can **alternativelly** call the function as follows:

```
# Load DJIA stock prices (+ include info about each stock)
prices <- info %>% tq_get(get = "stock.prices", from = tStart, to = tEnd)
```

- Which joins the information in the tibble **info** to the downloaded prices of the stocks (we can alternativelly do that also separatelly with left_join function)

Compute average annual returns

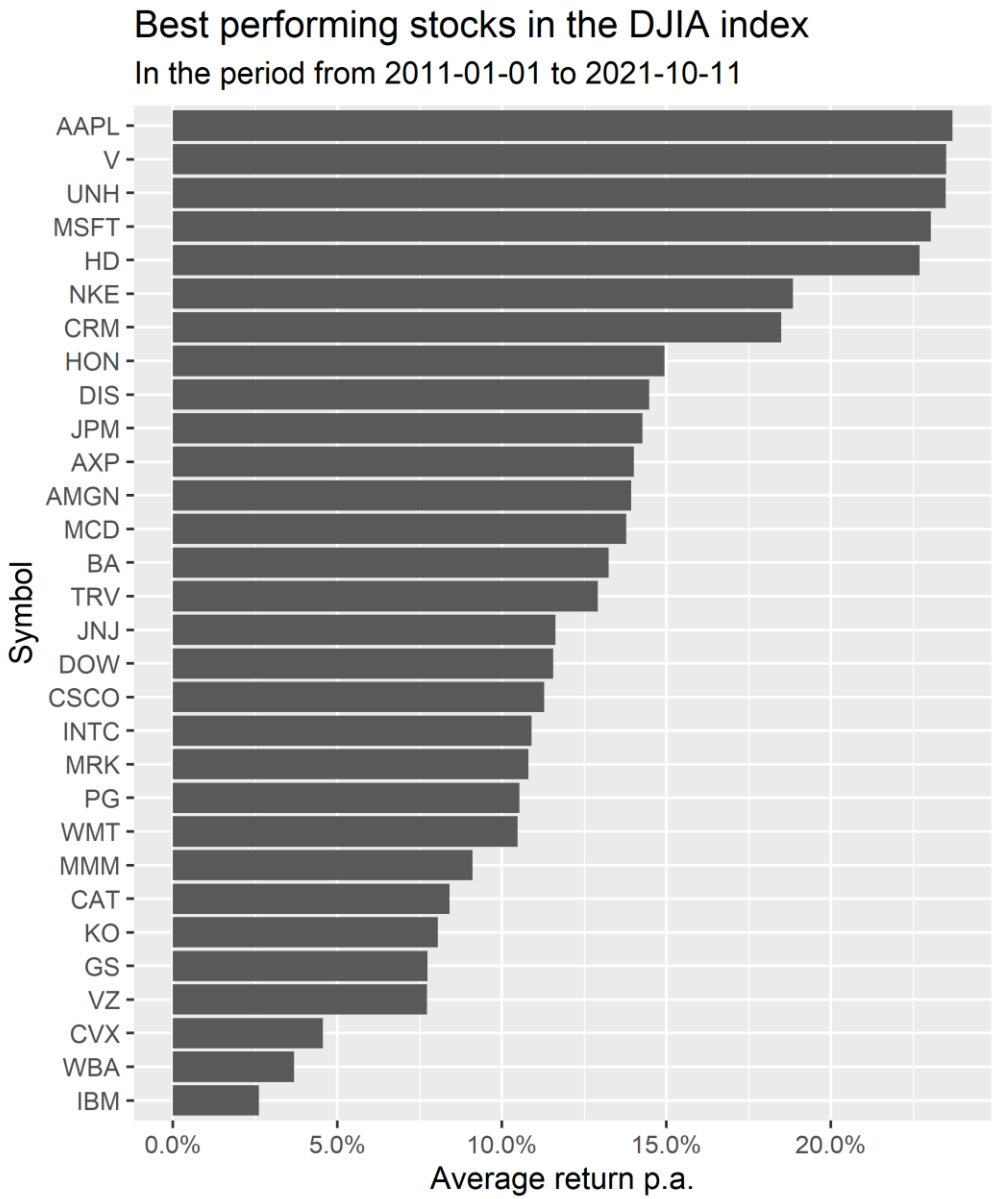
- Let us now compute the **average annual returns** for all of the stocks in the period since 1.1.2001 until today. We can do that as follows:

```
# Compute average yearly returns for all of the stocks:
stats <- prices %>%
  mutate(year = year(date)) %>%
  group_by(symbol,year) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return)) %>%
  summarize(return_pa = sum(return)) %>%
  ungroup() %>%
  group_by(symbol) %>%
  summarize(avg_return_pa = mean(return_pa))
```

- To plot the returns on a chart, we need to use **geom_col()** in the ggplot settings

```
# Plot the average annual returns on chart
stats %>%
  ggplot(aes(x = reorder(symbol,avg_return_pa), y = avg_return_pa)) +
  geom_col() +
  labs(title = paste0("Best performing stocks in the DJIA index"),
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       y = "Average return p.a.", x = "Symbol", color = "") +
  scale_y_continuous(labels = scales::percent) +
  coord_flip()
```

DJIA stocks – Average returns p.a.



- We can see that **AAPL** (Apple) was the best performing stock from the DJIA index in the period 2011-2021, achieving a return of > 25% per year
- Similarly, **IBM** was the worst performin, but it still attained a positive return during the period 2011-2021, with average return of ~2.5% per year

Compute average annual returns

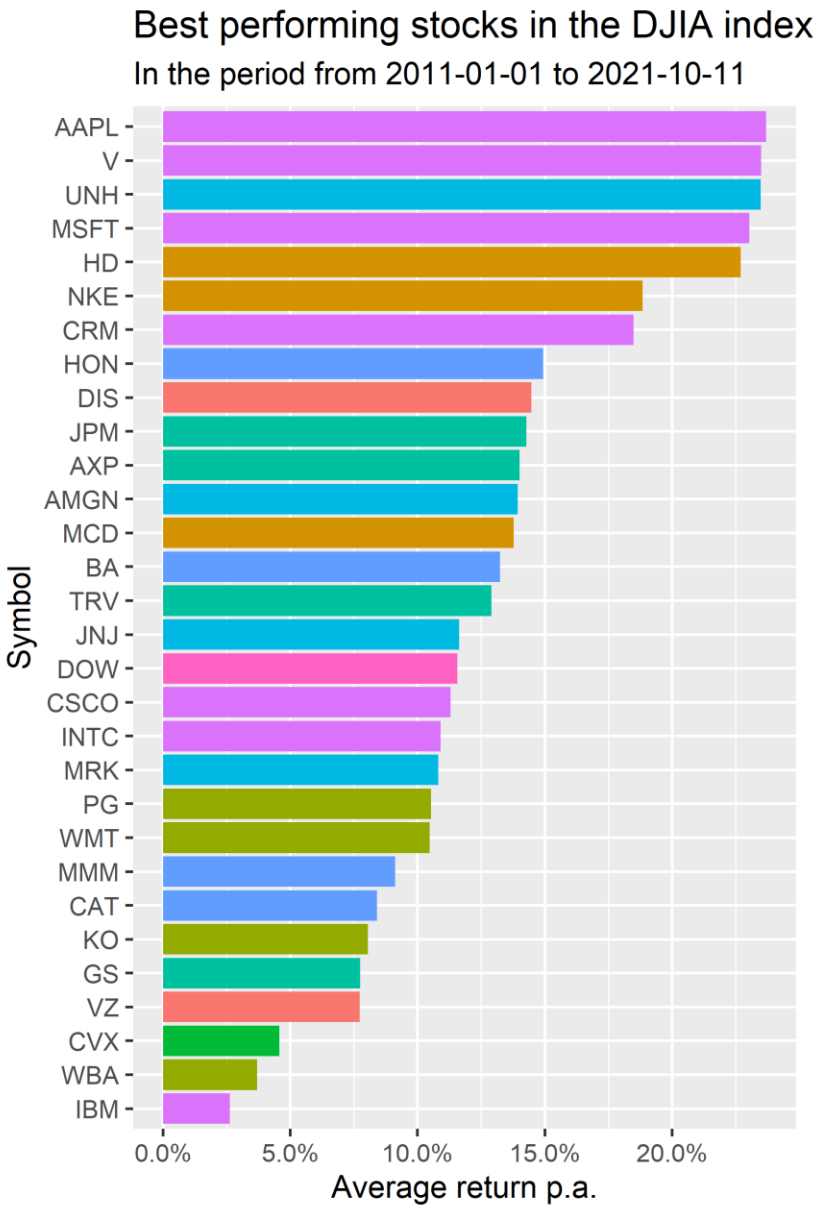
- We can also easily add information about the **sector** of the stock to the chart, by only slightly modifying the previous code:

```
# Compute average yearly returns for all of the stocks (+ sector):
stats <- prices %>%
  mutate(year = year(date)) %>%
  group_by(sector,symbol,year) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return)) %>%
  summarize(return_pa = sum(return)) %>%
  ungroup() %>%
  group_by(sector,symbol) %>%
  summarize(avg_return_pa = mean(return_pa))
```

- To plot the returns on a chart, we need to use **geom_col()** in the ggplot settings

```
# Plot the average annual returns on chart (+ sector)
stats %>%
  ggplot(aes(x = reorder(symbol,avg_return_pa), y = avg_return_pa, fill = sector)) +
  geom_col() +
  labs(title = paste0("Best performing stocks in the DJIA index"),
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       y = "Average return p.a.", x = "Symbol", color = "") +
  scale_y_continuous(labels = scales::percent) +
  coord_flip()
```

DJIA stocks – Average returns p.a.



- We can see that **Information Technology** stocks (AAPL, V, MSFT) are among the best performing stocks in the analyzed time period (2011-2021)

- Other **Information Technology** stocks did, however, underperform in the same time period (CSCO, INTC, IBM)

Compute average annual returns

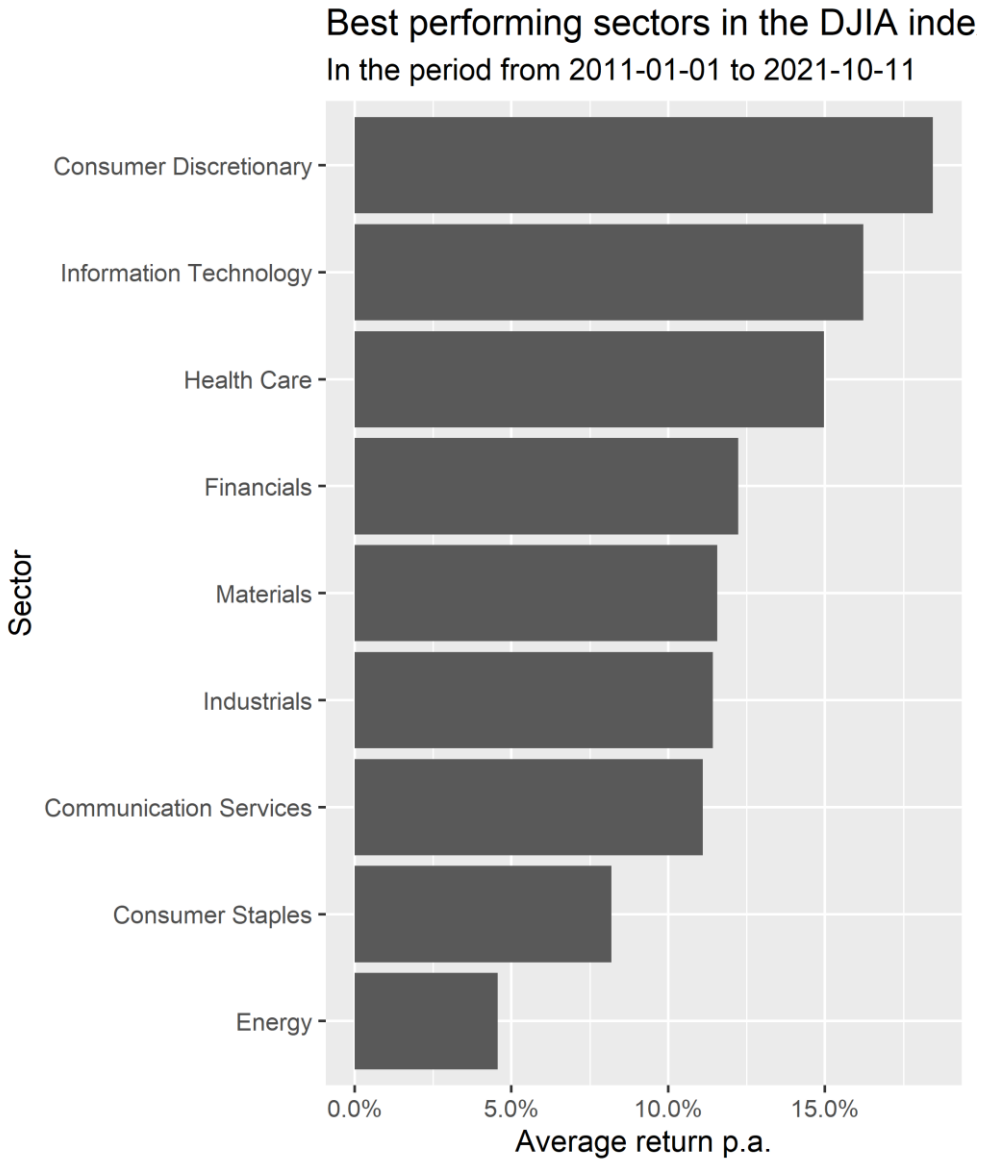
- Or compute the average annual returns of entire **sectors:**

```
# Compute average sector returns
stats <- prices %>%
  mutate(year = year(date)) %>%
  group_by(sector,symbol,year) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return)) %>%
  summarize(return_pa = sum(return)) %>%
  ungroup() %>%
  group_by(sector) %>%
  summarize(avg_return_pa = mean(return_pa))
```

- To plot the returns on a chart, we need to use **geom_col()** in the ggplot settings

```
# Plot the average annual sector returns on chart
stats %>%
  ggplot(aes(x = reorder(sector,avg_return_pa), y = avg_return_pa)) +
  geom_col() +
  labs(title = paste0("Best performing sectors in the DJIA index"),
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       y = "Average return p.a.", x = "Sector", color = "") +
  scale_y_continuous(labels = scales::percent) +
  coord_flip()
```

DJIA stocks – Average returns p.a.



- Overall, the **Consumer Discretionary** sector achieved the best returns (~18% p.a.) in the analyzed time period, followed by the **Information Technology** sector (~16% p.a.)
- The worst-performing sector was **Energy** (< 5% p.a.), which, however, contains only a single company (CVX) (Chevron Corporation)
- Take note that the results (as well as the others in this section) are influenced by the **look-forward bias** as we are working with the **current DJIA components** inctead of the **DJIA components in 2011 (!)**

- We can also analyse the stock returns over **multiple samples**:

```
# Average yearly returns in-sample vs. out-sample:
stats <- prices %>%
  mutate(year = year(date),
         sample = ifelse(year<=2015,"InSample","OutSample")) %>%
  group_by(sector,symbol,year,sample) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return)) %>%
  summarize(return_pa = sum(return)) %>%
  ungroup() %>%
  group_by(sector,symbol,sample) %>%
  summarize(avg_return_pa = mean(return_pa))
```

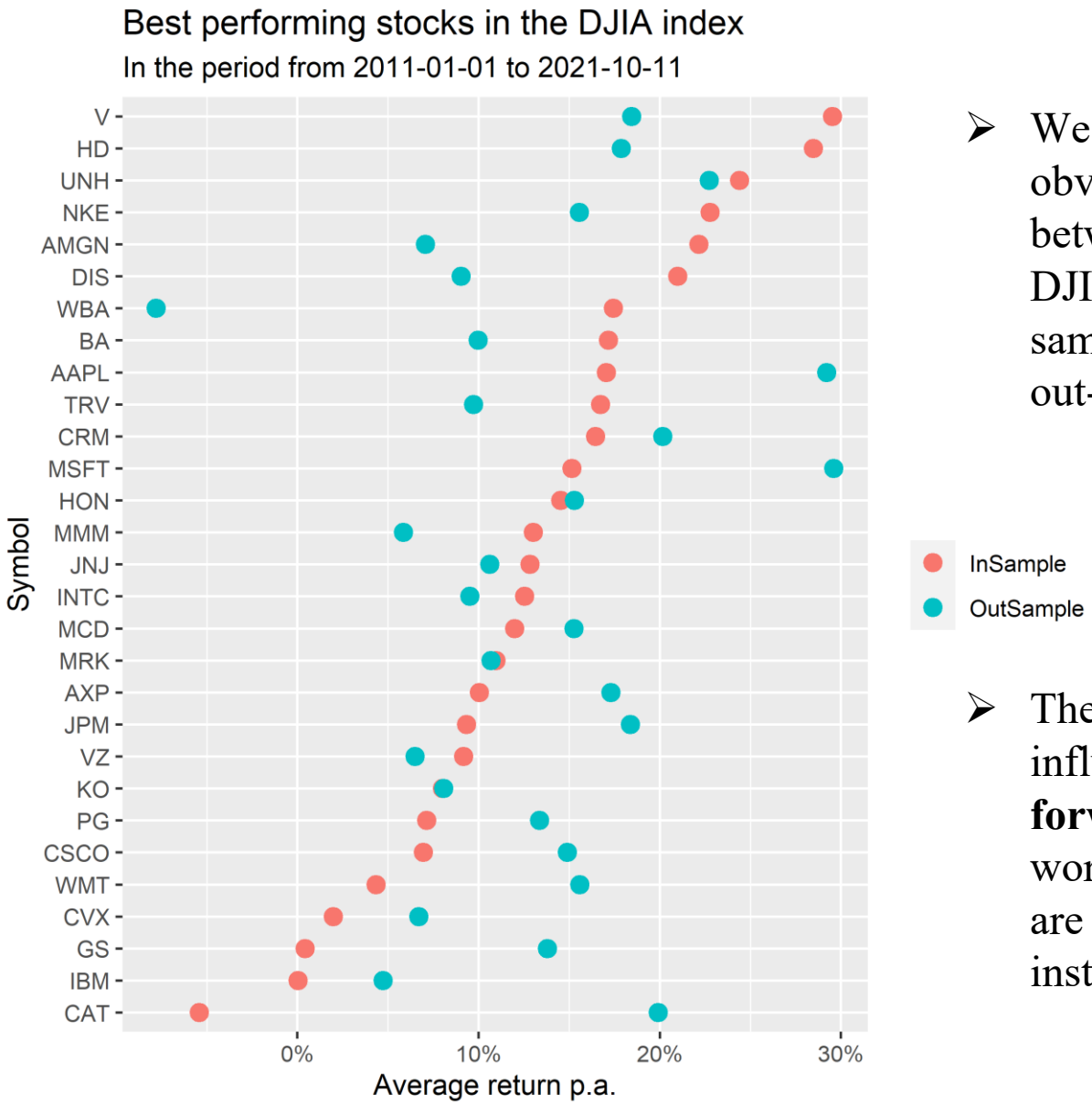
- To plot the returns on a chart, we need to use **geom_col()** in the ggplot settings

```
# Join the in-sample result to the table
temp <- stats %>%
  filter(sample == "InSample") %>%
  mutate(criterion = avg_return_pa) %>%
  ungroup() %>%
  select(symbol, criterion)
stats <- stats %>%
  left_join(temp, by="symbol")
```

- To plot the returns on a chart, we need to use **geom_col()** in the ggplot settings

```
# Plot average yearly returns in-sample vs. out-sample:
c25 <- stats %>%
  filter(is.na(criterion)==0) %>%
  ggplot(aes(x = reorder(symbol,criterion), y = avg_return_pa, color = sample)) +
  geom_point(size=3) +
  labs(title = paste0("Best performing stocks in the DJIA index"),
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       y = "Average return p.a.", x = "Symbol", color = "") +
  scale_y_continuous(labels = scales::percent) +
  coord_flip()
```

DJIA stocks – In-sample vs. Out-sample



- We can see that there is no obvious relationship between the returns of the DJIA stocks in the in-sample (2011-2015) and out-sample (2016-2021)

- The results are also still influenced by the **look-forward bias** as we are still working with stocks that are in the DJIA index now instead of in 2011 and 2016

Simple factor investing strategy

- **Task:** Let us now assess what would be the outcome of investing on an **annual basis** into portfolios composed of DJIA stocks that had alternativelly the **highest/lowest returns**, **volatility**, **skewness** and **volume** in the previous year (with annual rebalancing), in the years 2011-2021
- **Specifically:** At the start of each year, we will sort the stocks in the DJIA index into 5 quantile portfolio (1=lowest, 5=highest) based on the values of their returns, volatility, skewness and volume over the previous year, and then compute these portfolios in the current year in oder to assess how each of the predictors impacts the future returns
- **Summary of the steps:**

1. **Data download:** Download the DJIA index components with **tq_index** and the stock prices with **tq_get**
2. **Compute daily returns and USD volumes:** Compute daily log-returns and USD volumes with **mutate**
3. **Compute annual values of the factors (predictors):** Compute (i) yearly returns, (ii) yearly volumes, (iii) return volatility for each year, and (iv) return skewness for each year, with **summarize**
4. **Compute lagged values of the factors:** Compute lagged (i.e. last year) values of the 4 factors with **mutate**
5. **Remove rows with missing observations:** Remove year 2011 for which the lagged values will be NA
6. **Gather the predictors into a single column:** Gather all of the predictors into a single column with **gather**
7. **Assign cross-sectional ranks:** Assign to each stock its relative rank, based on the value of the given predictor in a given year compared to the values of that predictor in that year for the other stocks (relative rank of 0 means that the value of the predictor is the lowest and 1 that it is the highest among all stocks)
8. **Construct quintile portfolios:** Construct the quintile portfolios (1=20% of stocks with highest values of the predictor in the given year, 5= 20% of stocks with the lowest values of the predictor in the given year)
9. **Compute quintile portfolio returns:** Compute the returns of the quintile portfolios one-year ahead
10. **Compute average returns of the quintile portfolios p.a.:** Compute the average logarithmic returns p.a.
11. **Plot the average returns on a chart:** Plot the result on a chart with **ggplot**

Simple factor investing strategy (1)

- **Step 1 – Data download:**
- First, we downalod the **components** of the DJIA stock index with **tq_index:**

```
# Get DJIA components
info <- tq_index("DOW")
```

- We then set the remaining **settings** of the task:

```
# Settings:
tStart <- "2011-01-01"
tEnd   <- today()
```

- And we download the OHLCV prices with **tq_get:**

```
# Load DJIA stock prices (+ include info about each stock)
prices <- info %>% tq_get(get = "stock.prices", from = tStart, to = tEnd)
```

Simple factor investing strategy (2)

- **Step 2 – Compute daily returns and USD volumes:**

```
# Compute daily log-returns and USD volumes
prices <- prices %>%
  group_by(symbol) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return),
         volumeUSD = volume*close)
```

- **Step 3 – Compute annual values of the factors (predictors):**

```
# Compute annual statistics
yprices <- prices %>%
  ungroup() %>%
  mutate(year = year(date)) %>%
  group_by(symbol,year) %>%
  summarize(volatility = sd(return),
            skewness = skewness(return),
            return = sum(return),
            volumeUSD = sum(volumeUSD))
```

Note: We compute the volatility and skewness first, as the row `return = sum(return)` rewrites what is referenced by „return“ in the code

Simple factor investing strategy (3)

- **Step 4 – Compute lagged values of the factors:**

```
# Compute lagged values of the factors
yprices <- yprices %>%
  ungroup() %>%
  group_by(symbol) %>%
  mutate(lag_return = lag(return),
         lag_volume = lag(volumeUSD),
         lag_skewness = lag(skewness),
         lag_volatility = lag(volatility))
```

- **Step 5 – Remove rows with missing observations:**

```
# Remove rows with missing observations
yprices <- yprices[complete.cases(yprices),]
```

- Note: This will remove the observations (year 2011) for which we could not compute the lagged values (**warning:** take care when using this command as it may remove rows containing missing values that you do not want to remove)

- **Step 6 – Gather the predictors into a single column:**

```
# Gather the predictors
tprices <- yprices %>%
  gather(factor,value,lag_return:lag_volatility)
```

Simple factor investing strategy (4)

- **Step 7 – Assign cross-sectional ranks:**

```
# Assign cross-sectional ranks
tprices <- tprices %>%
  ungroup() %>%
  group_by(year,factor) %>%
  mutate(factor_rank = rank(value),
         relative_rank = (factor_rank-1)/(max(factor_rank)-1),
         relative_rank = ifelse(relative_rank==0,1/1000000,relative_rank))
```

Note: The relative rank would be equal to 1 for the given stock, if the value of the factor was highest for the given year (among all of the stocks), and it will be equal to 0 if the value of the factor was the lowest for the given year (among all of the stocks) (0 is adjusted to 1/1M for technical reasons)

- **Step 8 – Construct quintile portfolios:**

```
# Construct quintile portfolios
tprices <- tprices %>%
  mutate(quintile = ceiling(relative_rank*5))
```

- **Step 9 – Compute quintile portfolio returns:**

```
# Compute quintile portfolio returns
qreturns <- tprices %>%
  ungroup() %>%
  group_by(year,factor,quintile) %>%
  summarize(return = mean(return))
```

- **Example:** For the **lag_return** factor, the quantile portfolio **1** will contain in each year the stocks that were among the 20% of stocks with the lowest return in the previous year, and portfolio **5** will contain the stocks that were among the 20% of stocks with the highest return in the previous year (out of all of the stocks in DJIA)

Simple factor investing strategy (5)

- **Step 10 – Compute average returns of the quintile portfolios p.a.:**

```
# Compute average quintile portfolio returns p.a.
stats <- qreturns %>%
  ungroup() %>%
  group_by(factor,quintile) %>%
  summarize(return_pa = mean(return),
            return_total = sum(return),
            return_exp = exp(return_total)-1)
```

- **Step 11 – Plot the average returns on a chart:**

```
# Plot average average returns of the quintile portfolios
stats %>%
  group_by(factor,quintile) %>%
  ggplot(aes(x = quintile, y = return_pa, fill = factor)) +
  geom_col() +
  labs(title = "Quintile portfolio average returns",
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       x = "",y = "Average return") +
  facet_wrap(~ factor, ncol = 2, scales = "free_y") +
  scale_y_continuous(labels = scales::percent)
```

Simple factor investing strategy (6)

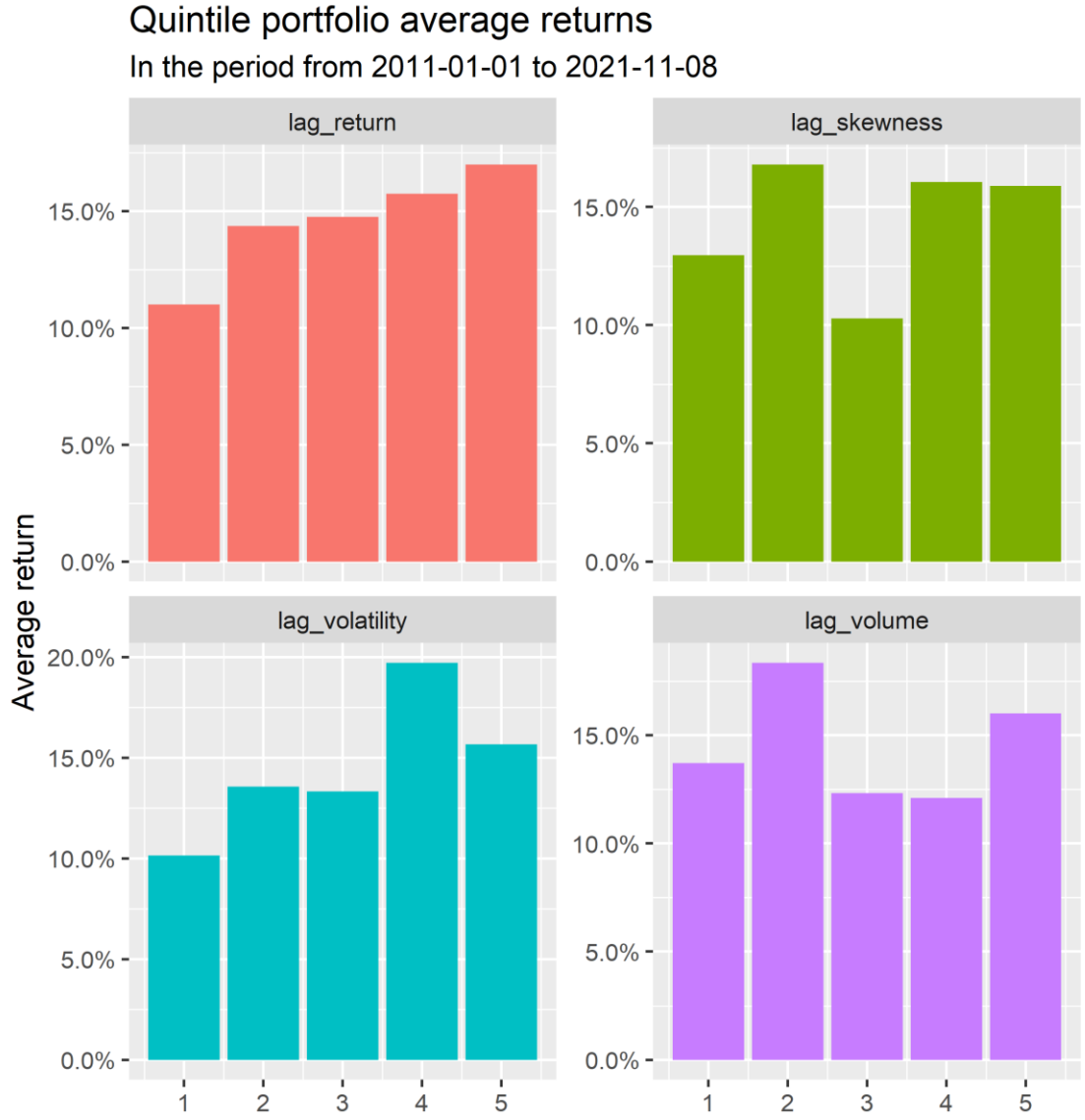
- **Step 10 – Compute average returns of the quintile portfolios p.a.:**

```
# Compute average quintile portfolio returns p.a.
stats <- qreturns %>%
  ungroup() %>%
  group_by(factor,quintile) %>%
  summarize(return_pa = mean(return),
            return_total = sum(return),
            return_exp = exp(return_total)-1)
```

- **Step 11 – Plot the average returns on a chart**

```
# Plot average average returns of the quintile portfolios
stats %>%
  group_by(factor,quintile) %>%
  ggplot(aes(x = quintile, y = return_pa, fill = factor)) +
  geom_col() +
  labs(title = "Quintile portfolio average returns",
       subtitle = paste0("In the period from ",tStart," to ",tEnd),
       x = "",y = "Average return") +
  facet_wrap(~ factor, ncol = 2, scales = "free_y") +
  scale_y_continuous(labels = scales::percent)
```

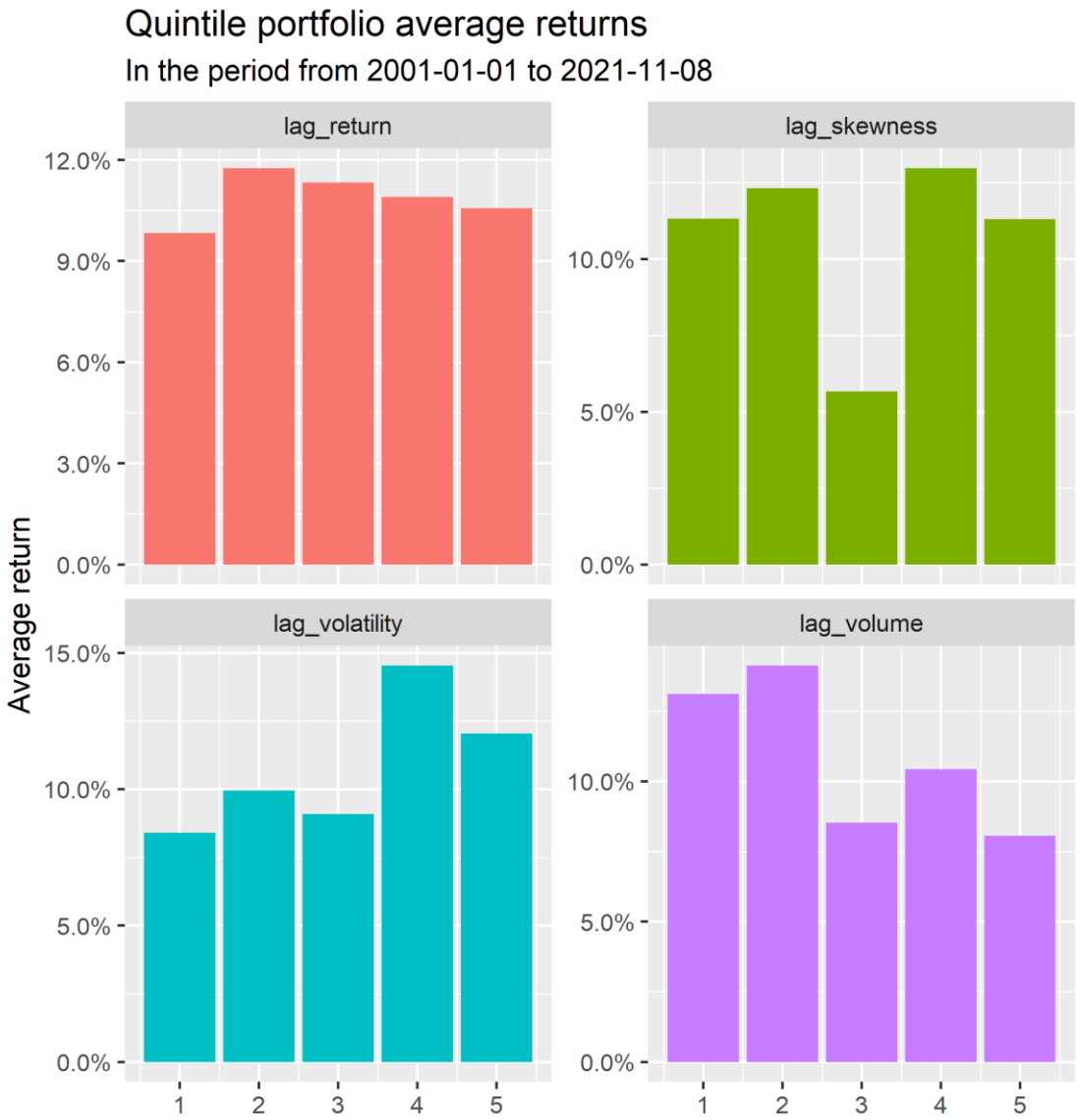
Factor investing strategy - Results



- In years 2011-2021 it would be profitable to invest into DJIA stocks (with annual rebalancing) that had:
  - Highest returns in previous year
  - High, but not highest volatility in the previous year

- Take note, however, that the results are (i) probably not statistically significant, (ii) in-sample, (iii) suffering from look-forward bias as we are working with the ex-post components of DJIA (in 2021)

Recalculation on the data since 2001



- We can see that by merely prolonging the dataset to 2001-2021 changed the results significantly and the lagged volume suddenly has a negative relationship with subsequent returns

- Note: Investing strategies of this kind are called **characteristic-based factor investing strategies** and we will look at how to develop them more rigorously later during this course